



Universidad de
Oviedo



ESCUELA POLITÉCNICA DE INGENIERÍA DE GIJÓN

MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA

**ÁREA DE ARQUITECTURA Y
TECNOLOGÍA DE COMPUTADORES**

TRABAJO FIN DE MÁSTER

*Optimización interactiva del contraste de imagen
en flujos de captura cruda para procesamiento en
tiempo real*

D. Rubén Martínez Ginzo

TUTOR: *D. Rubén Usamentiaga Fernández*

FECHA: enero 2026

Índice

1	Introducción	7
2	Objetivo y alcance	8
2.1	Estudio del estado del arte	8
2.2	Aplicación interactiva	9
2.3	Programa de <i>benchmarking</i>	9
2.4	Análisis de resultados	9
3	Aspectos teóricos	10
3.1	Imágenes e Informática	10
3.2	Representación numérica de imágenes digitales	11
3.2.1	Muestreo	11
3.2.2	Cuantificación	11
3.3	Espacios de color	13
3.3.1	RGB	13
3.3.2	HSV	15
3.3.3	Otros	16
3.4	Formatos	17
3.4.1	Imágenes en formato crudo	17
3.5	Fundamentos del Procesamiento Digital de Imágenes	19
3.5.1	Operaciones puntuales y operaciones espaciales	19
3.5.2	El histograma como herramienta de diagnóstico	21
3.5.3	Mejora del contraste	22
3.5.4	Filtrado espacial y reducción de ruido	29
3.5.5	Realce de detalle y nitidez	32
3.5.6	Otras operaciones	34
3.5.7	El <i>pipeline</i> de revelado digital	38
3.6	Representación computacional de imágenes digitales	39
3.6.1	<i>PyTorch</i> como <i>framework</i> para el procesamiento de imágenes	39
3.6.2	Tensores en la representación de imágenes	39
3.6.3	Organización de datos: canales y dimensiones	40
3.6.4	Tipos de datos y precisión numérica	41
3.7	Procesamiento en tiempo real	42
3.8	Computación acelerada por GPU	43
4	Estudios previos y análisis de alternativas	44
5	Planificación	45

6	Presupuesto	46
7	Análisis	47
7.1	Requisitos de usuario	47
7.2	Mapa de historias de usuario	47
7.3	Prototipos de interfaz de usuario	47
7.4	Mapa de navegación	47
8	Diseño	48
8.1	Visión general	48
8.1.1	Principios de diseño	48
8.1.2	Flujo de ejecución	48
8.2	Arquitectura	49
8.2.1	Enfoque arquitectónico	49
8.3	Componentes	50
8.3.1	Models	50
8.3.2	Loaders	52
8.3.3	Core	53
8.3.4	Controller	55
8.3.5	GUI	57
8.3.6	Benchmark	66
8.3.7	Tests	68
8.4	Diagrama de paquetes	70
9	Benchmarking	71
10	Resultados	72
11	Pruebas	73
12	Conclusiones	74
A	Manual de usuario	75
B	Stack tecnológico	76
C	Formatos de imagen cruda	77

Índice de figuras

1	Proceso de digitalización de una imagen	12
2	Cubo de color RGB	13
3	Descomposición de canales RGB	14
4	Cono de color HSV	15
5	Descomposición de componentes HSV	16
6	Patrones Bayer más comunes utilizados en sensores de imagen digitales	18
7	Imagen y su correspondiente histograma	21
8	Ejemplos de imágenes con diferentes niveles de contraste	22
9	Ejemplo de aplicación de ecualización global del histograma	23
10	Ejemplo de aplicación de CLAHE	24
11	Ejemplo de aplicación de normalización min-max	25
12	Ejemplo de aplicación de normalización min-max con percentiles	26
13	Ejemplo de corrección gamma	27
14	Ejemplo de mejora del contraste mediante función sigmoide	28
15	Ejemplo de aplicación de filtro gaussiano	30
16	Ejemplo de aplicación de filtro de mediana	31
17	Ejemplo de aplicación de <i>Unsharp Masking</i>	33
18	Ejemplo de aplicación de volteo	34
19	Ejemplo de aplicación de rotación	35
20	Etapas del <i>pipeline</i> de revelado digital	38
21	Representación visual de tensores de diferentes dimensiones	40
22	Representación conceptual de una imagen RGB como tensor tridimensional	40
23	Ejemplo de representación tensorial de una imagen RGB de 3×3 píxeles	41
24	Comparación conceptual entre arquitecturas CPU y GPU	43
25	Flujo general de ejecución del sistema	48
26	Diagrama de arquitectura del sistema	49
27	Sistema de <i>snapshots</i> para optimización del <i>pipeline</i> de operaciones	56
28	Visión general de la interfaz gráfica (sin imagen cargada)	57
29	Visión general de la interfaz gráfica (con imagen cargada)	57
30	Panel izquierdo (<i>LeftPanel</i>)	58
31	Gráficos de información de la imagen	58
32	Botones de guardado y reinicio	59
33	Selector de operaciones	59
34	Botones para generar el perfil de ejecución de operaciones y para guardar la imagen procesada	59
35	Visor central (<i>ImageViewer</i>)	60
36	Sección de <i>tabs</i> del visor central	60
37	Pestaña de <i>Logger</i> del visor central	61
38	Cambio de dispositivo de procesamiento y referencia de tiempos de ejecución	61

39	Pestaña de <i>Device Info</i> del visor central	61
40	Panel derecho (<i>RightPanel</i>)	62
41	Controles para operaciones con selección de método o algoritmo	62
42	Controles para operaciones con parámetros numéricos ajustables	63
43	Controles para operaciones con parámetros numéricos ajustables (continuación)	64
44	Control para Volteo de imagen	64
45	Control para Compensación de Iluminación	64
46	Controles para operaciones sin parámetros	65

Índice de tablas

Índice de fragmentos

1 Introducción

[...]

Trabajo de Fin de Máster (TFM)

2 Objetivo y alcance

Este TFM consiste en el planteamiento y desarrollo de una aplicación interactiva para la optimización del contraste de imágenes en formato crudo, orientada a su aplicación en flujos de procesamiento en tiempo real acelerado por GPU.

La aplicación permitirá al usuario ajustar de manera visual e interactiva los parámetros de revelado y realce de contraste sobre imágenes crudas, visualizando en tiempo real el efecto de cada operación sobre la imagen. A partir de dicha interacción, el sistema permitirá generar un perfil de procesamiento reproducible, que describa de forma declarativa una secuencia de operaciones de imagen y sus parámetros asociados. Este perfil podrá ser exportado y aplicado en entornos de procesamiento en tiempo real, garantizando:

- Coherencia visual entre la etapa de diseño interactivo y la ejecución en producción.
- Eficiencia computacional mediante el uso de procesamiento vectorizado y aceleración por GPU.
- Separación entre la definición del perfil de procesamiento y su ejecución.

Además, se desarrollará un programa de *benchmarking* para evaluar el rendimiento de diferentes perfiles de procesamiento y operaciones realizadas sobre imágenes crudas en términos de velocidad de ejecución. Este programa permitirá comparar ejecuciones de perfiles en diferentes sistemas y entornos, dispositivos de procesamiento (CPU vs GPU) y formatos de tratamiento de imágenes. Para lograr todos estos objetivos, este TFM se divide en varias etapas, las cuales se describen a continuación.

2.1 Estudio del estado del arte

En esta etapa se realizará una revisión exhaustiva de la literatura y tecnologías existentes relacionadas con el procesamiento de imágenes en formato crudo, técnicas de optimización de contraste y herramientas de desarrollo que se enmarquen en este contexto. Debe prestarse especial atención a las siguiente áreas:

- **Procesamiento de imágenes en formato crudo.** Se explorarán las características y particularidades de los formatos de imagen cruda más comunes, así como los algoritmos y técnicas empleadas para su revelado y posprocesamiento.
- **Optimización de contraste.** Se investigarán los métodos y algoritmos utilizados para mejorar el contraste de imágenes, incluyendo técnicas clásicas y enfoques más recientes.
- **Herramientas y bibliotecas de desarrollo.** Se analizarán las herramientas, bibliotecas y frameworks disponibles, evaluando su idoneidad para los objetivos del TFM.
- **Aplicaciones prácticas y estudios de caso.** Se revisarán y explorarán aplicaciones prácticas ya existentes que aborden problemas similares, analizando su enfoque, implementación y tecnologías.

2.2 Aplicación interactiva

En esta fase se abordará el diseño de la arquitectura de software de la aplicación, definiendo los componentes necesarios para la carga de imágenes, la aplicación de operaciones de procesamiento y la visualización interactiva de resultados. Deberá de diferenciarse claramente entre los siguientes módulos:

- **Módulo de carga y gestión de imágenes crudas.** Responsable de la lectura y almacenamiento eficiente de imágenes en formato crudo, así como de su conversión a formatos adecuados para el procesamiento.
- **Módulo de procesamiento de imágenes.** Implementará las operaciones de revelado y optimización de contraste. Deberán de definirse de forma que sean completamente independientes, parametrizables y combinables en secuencias de procesamiento. Además, se implementarán pruebas unitarias para garantizar su correcto funcionamiento.
- **Módulo de interfaz gráfica.** Proporcionará un entorno interactivo e intuitivo para el flujo de trabajo de procesamiento. Permitirá a los usuarios aplicar operaciones, ajustar parámetros y visualizar resultados en tiempo real sobre la imagen cargada. Además, incluirá la funcionalidad de exportación del perfil de procesamiento generado.

2.3 Programa de *benchmarking*

En esta etapa se desarrollará un programa específico para la evaluación del rendimiento de los *pipelines* de procesamiento definidos. Dicho programa permitirá ejecutar perfiles sobre diferentes tamaños de imagen, dispositivos de cómputo y configuraciones, registrando métricas de tiempo de ejecución y escalabilidad. Será de gran importancia que el programa facilite la toma y recogida de datos de rendimiento de manera estructurada para su posterior visualización y análisis.

2.4 Análisis de resultados

En esta última fase se llevará a cabo el análisis e interpretación de los resultados obtenidos a partir del programa de *benchmarking*. Se estudiará el comportamiento temporal de los diferentes perfiles de procesamiento, evaluando cómo influyen factores como el tamaño de la imagen, el número de operaciones encadenadas y el dispositivo de ejecución utilizado.

3 Aspectos teóricos

Se plantean y describen en este apartado los aspectos teóricos y tecnológicos que sustentan el desarrollo del presente TFM. Estos conceptos derivan directamente del estudio del estado del arte y son necesarios para comprender las decisiones de diseño, los enfoques de implementación y los criterios de evaluación que se exponen en los capítulos posteriores.

3.1 Imágenes e Informática

La Imagen ha sido, históricamente, uno de los principales medios para la representación, el registro y la comunicación de la realidad. Su valor informativo reside en la capacidad de codificar la información espacial y tonal de una escena u objeto, actuando como nexo entre la percepción visual humana y los procesos de interpretación cognitiva.

Desde un punto de vista teórico, y con anterioridad a su tratamiento computacional, una imagen puede modelarse como una función bidimensional continua de intensidad o color. En este dominio analógico, la imagen representa una escena asociando a cada punto del plano (x, y) un valor de intensidad luminosa. Esta relación puede expresarse matemáticamente como se muestra en (1):

$$I = f(x, y) \quad \text{donde } (x, y) \in \mathbb{R}^2 \text{ e } I \in \mathbb{R} \quad (1)$$

En este contexto pre-digital, la calidad de la imagen depende fundamentalmente de las características físicas del sistema de captura, tales como la óptica empleada, las condiciones de iluminación o la naturaleza de la emulsión química. Tradicionalmente, la captura de imágenes se realizaba mediante procesos fotoquímicos, en los que la luz incidente impresionaba una emulsión fotosensible sobre soportes físicos como placas o películas fotográficas. Este tipo de representación tiene una gran limitación: una vez capturada, la capacidad de procesar, almacenar o transmitir una imagen analógica resulta muy limitada.

La incorporación de tecnologías de captura electrónica y digital, tales como los sensores CCD (*Charge-Coupled Device*) y CMOS (*Complementary Metal-Oxide-Semiconductor*) [1], marca un cambio de paradigma. La imagen deja de ser una representación puramente analógica para convertirse en una entidad digital, susceptible de ser manipulada, almacenada y transmitida mediante sistemas informáticos. Es en este punto donde se produce la intersección directa entre la Imagen y la Informática. Intersección que da lugar al Procesamiento Digital de Imágenes (PDI), uno de los campos más dinámicos y de mayor impacto en la era digital actual [2].

Para que una imagen pueda ser procesada y tratada por un sistema informático, debe abandonar su naturaleza continua y transformarse en el único lenguaje que la máquina es capaz de interpretar: el dato numérico.

3.2 Representación numérica de imágenes digitales

La captura de una imagen digital implica un proceso de digitalización que convierte la función continua mostrada en la Ecuación 1 en una representación discreta y cuantificada. Dicho proceso consta de dos etapas principales: el muestreo y la cuantificación.

3.2.1 Muestreo

El muestreo consiste en la discretización del dominio continuo de la imagen, dividiendo el plano (x, y) en una rejilla regular de posiciones finitas: los *píxeles*. Cada píxel representa una pequeña región de la escena original y constituye la unidad mínima de información espacial en una imagen digital. El resultado de este proceso es una imagen discreta definida sobre un conjunto finito de coordenadas enteras (2):

$$I[i, j] = f(i\Delta x, j\Delta y) \quad (2)$$

donde (i, j) representan las coordenadas del píxel y $\Delta x, \Delta y$ las resoluciones espaciales en cada dimensión. El número total de muestras en cada dimensión determina la resolución espacial de la imagen y establece un compromiso entre el nivel de detalle representable y los recursos necesarios para su almacenamiento y procesamiento.

3.2.2 Cuantificación

La cuantificación, por su parte, es el proceso mediante el cual los valores continuos de intensidad asociados a cada píxel se aproximan a un conjunto finito de niveles discretos. Mientras que el muestreo actúa sobre el dominio espacial de la imagen, la cuantificación opera sobre su dominio de amplitud, esto es, los valores numéricos que representan la intensidad luminosa o el color en cada píxel.

La cuantificación puede modelarse como una función no lineal $Q(\cdot)$ que asigna a cada valor continuo uno de los valores discretos disponibles (3):

$$I[i, j] = Q(f(i, j)) \quad (3)$$

El conjunto de valores posibles tras la cuantificación está determinado por la profundidad de bits (*bit depth*) b del sistema de representación, que define un total de $L = 2^b$ niveles distintos. Este valor determina la precisión con la que se puede representar la intensidad de cada píxel, afectando directamente a la calidad visual de la imagen resultante.

El proceso de digitalización se ilustra de forma esquemática en la Figura 1, donde se muestra la transición desde una imagen continua $f(x, y)$, pasando por la imagen muestreada $f[m, n]$ hasta la imagen digital final $I[m, n]$.

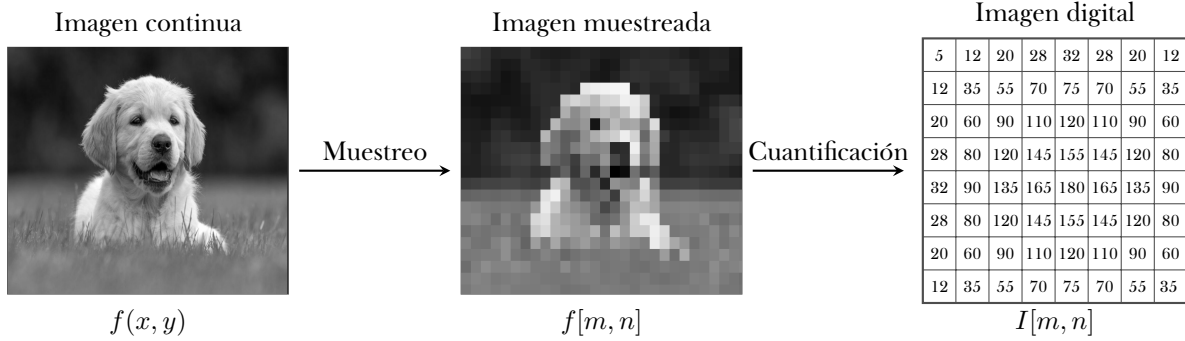


Figura 1: Proceso de digitalización de una imagen

El resultado final de este proceso es la imagen digital, que se representa como una matriz bidimensional ($M \times N$) de valores discretos. Cada elemento de esta matriz corresponde a un píxel con un valor cuantificado que refleja la intensidad luminosa o color en esa posición específica (4).

$$I[m, n] \quad \text{donde } m = 0, 1, \dots, M - 1 \text{ y } n = 0, 1, \dots, N - 1 \quad (4)$$

No obstante, la forma concreta que adopta esta representación matricial depende de la naturaleza cromática de la imagen. En el caso de imágenes en escala de grises, tal y como mostró en la Figura 1, cada píxel queda descrito por un único valor de intensidad $i \in [0, I]$, donde I es el valor máximo determinado por la profundidad de bits (5).

$$\text{Imagen Gris} \rightarrow I \in R^{M \times N} \quad (5)$$

Por el contrario, en las imágenes a color, cada píxel se representa mediante múltiples componentes, generalmente organizadas en distintos canales que codifican la información cromática según un modelo de color determinado. En el modelo más común, el modelo RGB (*Red*, *Green*, *Blue*), cada píxel se define por tres valores de intensidad (6):

$$\text{Imagen Color (RGB)} \rightarrow I = (I_R, I_G, I_B) \quad \text{donde } I_R, I_G, I_B \in R^{M \times N} \quad (6)$$

El valor de color final de cada píxel específico (i, j) es un vector que combina los valores de los tres canales en esa posición: $I[i, j] = (I_R[i, j], I_G[i, j], I_B[i, j])$.

3.3 Espacios de color

Un espacio de color es un sistema específico que permite la representación matemática de los colores. Se basa en un modelo de color al que se le asigna una escala y rango definidos, permitiendo que cada color sea identificado de forma única mediante coordenadas numéricas [3]. Los colores se pueden representar en diferentes espacios de color y se pueden transformar de uno a otro sin que se pierda información cromática. En el contexto de este TFM, existen varios espacios de color que resultan relevantes.

3.3.1 RGB

El espacio de color RGB es el modelo aditivo más comúnmente utilizado en los sistemas de captura, representación y visualización de imágenes digitales. En este modelo, los colores se generan mediante la combinación de tres componentes primarias: Rojo (Red), Verde (Green) y Azul (Blue), cuyas intensidades determinan el color final percibido.

Desde el punto de vista de la representación numérica, cada píxel de una imagen en el espacio RGB queda definido por un triplete ordenado (R, G, B) , donde cada componente corresponde a la intensidad cuantificada de uno de los canales de color, tal como se mostró en la Ecuación 6. Estos valores suelen representarse en un rango discreto determinado por la profundidad de bits del sistema, siendo habitual el uso de 8 bits por canal, lo que permite representar hasta 256 niveles por componente y un total de más de 16 millones de combinaciones cromáticas posibles.

Geométricamente, el espacio RGB puede interpretarse como un cubo tridimensional en un sistema de coordenadas cartesianas, donde cada eje corresponde a uno de los canales de color. En esta representación, el origen $(0, 0, 0)$ corresponde al color negro, mientras que el vértice opuesto $(R_{\text{máx}}, G_{\text{máx}}, B_{\text{máx}})$ representa el color blanco. La Figura 2 ilustra esta distribución de los colores dentro del cubo RGB.

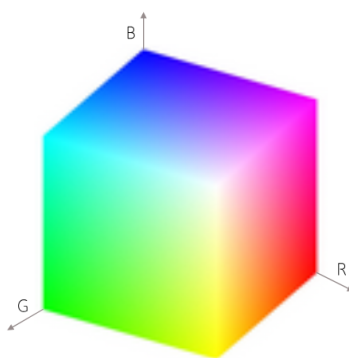
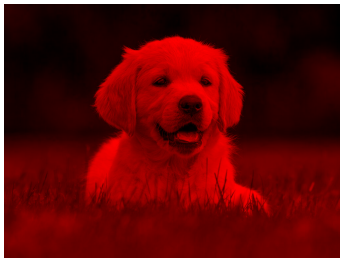


Figura 2: Cubo de color RGB
[4]

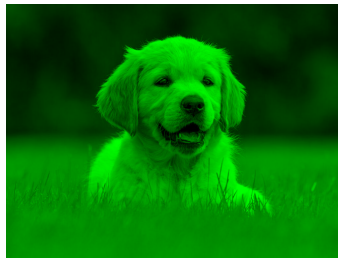
Con el objetivo de ilustrar de forma visual la representación de una imagen en el espacio de color RGB, la Figura 3 muestra un ejemplo de descomposición de una imagen a color en sus tres canales constituyentes. Cada canal almacena la información de intensidad correspondiente a una de las componentes primarias, dando lugar a tres imágenes monocromas independientes. La combinación de estos tres canales permite reconstruir la imagen original en color, ejemplificando la naturaleza de la representación RGB y su relación directa con la estructura matricial descrita anteriormente.



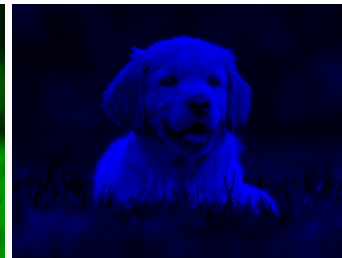
(a) Imagen original



(b) Canal R



(c) Canal G



(d) Canal B

Figura 3: Descomposición de canales RGB

3.3.2 HSV

El espacio de color HSV (*Hue, Saturation, Value*) es un modelo de representación cromática derivado del espacio RGB. Su objetivo es desacoplar la información de brillo de la información de color [5].

Al igual que en el modelo RGB, cada píxel de una imagen en el espacio HSV se representa mediante un vector tridimensional. No obstante, en este caso, las componentes se organizan de acuerdo con una interpretación más cercana a la percepción humana del color. La estructura geométrica del espacio HSV es cilíndrica, tal y como se muestra en la Figura 4, y separa la información visual en tres componentes independientes:

- **Hue (Tono, Matiz).** Representa el color puro y se mide en grados (0° a 360°) alrededor del cilindro. Cada valor de tono corresponde a un color específico (rojo, verde, azul, etc.).
- **Saturation (Saturación).** Define la pureza del color, es decir, qué tan diluido está con el blanco. Varía desde un tono gris (0 %) hasta el color más intenso (100 %).
- **Value (Valor, Brillo).** Representa el brillo del color, independientemente de su tono y saturación. Un 0 % representa la ausencia total de luz (negro).

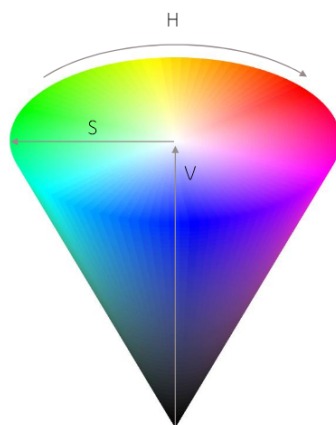


Figura 4: Cono de color HSV
[4]

Este espacio de color facilita la aplicación de técnicas de procesamiento que actúan selectivamente sobre el brillo o el color, permitiendo, por ejemplo, realizar ajustes de iluminación sin alterar la esencia cromática de la imagen. Esta es la principal diferencia respecto a lo que ocurre en el espacio RGB, donde la información de color y brillo se encuentra intrínsecamente entrelazada en sus tres canales.

De nuevo, con el objetivo de ilustrar de forma visual la representación de una imagen en el espacio de color HSV, la Figura 5 muestra un ejemplo de descomposición de una imagen a color en sus tres componentes constituyentes.

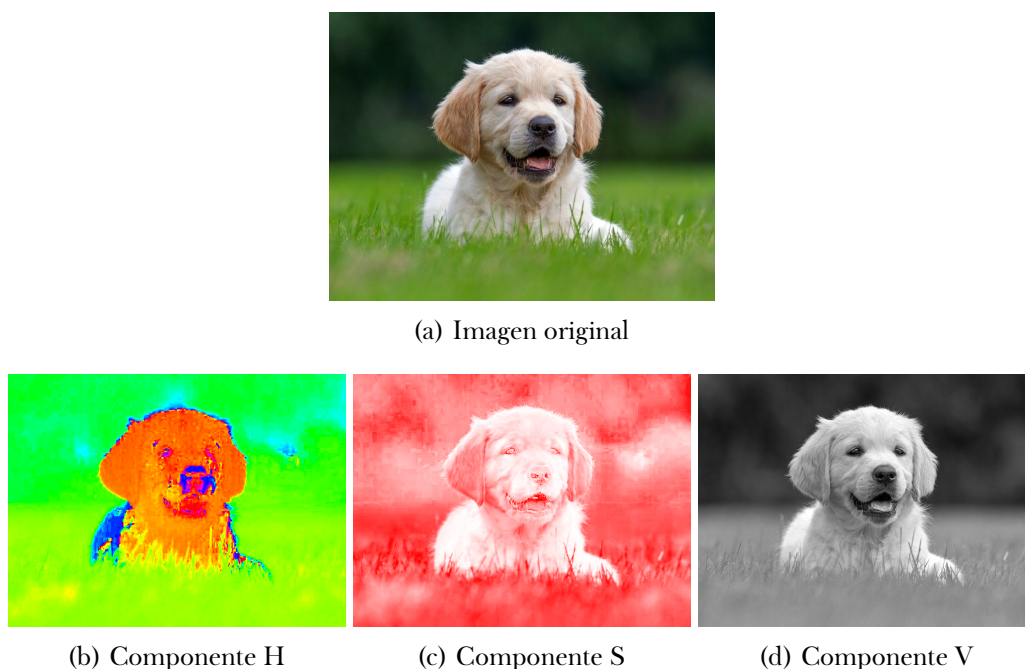


Figura 5: Descomposición de componentes HSV

3.3.3 Otros

Además de los ya mencionados, existen otros espacios de color ampliamente utilizados en el contexto del procesamiento digital de imágenes y la codificación visual que, aunque no se emplean en este TFM, resultan relevantes por sus propiedades particulares y su impacto en técnicas de análisis y compresión de imágenes

- **L*a*b***. Este espacio de color fue diseñado para aproximar la percepción visual humana, de modo que variaciones iguales en los valores de los canales se correspondan con cambios perceptibles similares en el color. L*a*b* separa de forma explícita la información de luminosidad (L^*) de las componentes cromáticas (a^* y b^*), donde a^* representa la variación entre verde y rojo, y b^* la variación entre azul y amarillo [6].
- **YCbCr**. Utilizado principalmente en compresión de imágenes y vídeo, como JPEG o MPEG, este espacio separa la luminancia (Y) de las componentes de cromaticidad (Cb y Cr), que codifican la diferencia de color respecto al brillo [7].

3.4 Formatos

Establecida la representación numérica de la imagen y su codificación cromática, es necesario determinar cómo se estructura y almacena dicha información. Los formatos de imagen definen la forma en que los datos de una imagen digital son almacenados, representados e intercambiados entre sistemas [8]. Un formato de imagen no sólo especifica la organización de los datos de píxeles, sino también la codificación del color, la profundidad de bits, los mecanismos de compresión y, en muchos casos, la inclusión de metadatos adicionales.

Desde el punto de vista de su representación geométrica, los formatos de imagen pueden clasificarse en dos grandes categorías: formatos *raster* y formatos *vectoriales*. Los formatos raster almacenan la imagen como un mapa de bits, es decir, como una matriz bidimensional de píxeles discretos, tal y como se ha descrito en los apartados anteriores. Por el contrario, los formatos vectoriales representan la información visual mediante primitivas geométricas, como líneas, curvas o polígonos, junto con los atributos asociados que definen su apariencia visual.

En el contexto del presente TFM, el interés se centra exclusivamente en los formatos raster, dado que constituyen la base de la captura y el procesamiento de imágenes digitales procedentes de sensores. Dentro de esta categoría existe una amplia variedad de formatos, entre los que se encuentran JPEG, PNG, BMP o TIFF, cada uno de ellos diseñado con objetivos específicos relacionados con la compresión, la calidad de imagen o la compatibilidad con distintos sistemas.

No obstante, para los fines de este trabajo, se presta especial atención a las imágenes en *formato crudo* (*RAW*), ya que representan la información capturada por el sensor con un nivel mínimo de procesamiento y sin pérdidas asociadas a técnicas de compresión o postprocesado. Este tipo de formato constituye la base sobre la que se desarrollan los métodos de procesamiento propuestos en este TFM, permitiendo un mayor control sobre la información radiométrica y cromática de la imagen.

3.4.1 Imágenes en formato crudo

Las imágenes en formato crudo (*RAW*), representan la forma más directa y fiel de los datos capturados por un sensor de imagen digital [9]. A diferencia de los formatos convencionales procesados y comprimidos, los formatos crudos almacenan los valores de intensidad registrados por el sensor con un procesamiento mínimo o nulo, preservando la información original de la escena capturada.

En una imagen RAW, cada valor almacenado corresponde (de forma aproximada) a la respuesta del sensor a la radiación luminosa incidente, normalmente con profundidades de entre 12 y 16 bits por píxel. Esto permite representar un rango dinámico considerablemente mayor que el de los formatos estándar de 8 bits, conservando detalles tanto en las zonas más oscuras como en las más iluminadas de la imagen. En la mayoría de los sensores de imagen empleados en cámaras digitales, cada elemento fotosensible es capaz de medir únicamente la intensidad luminosa, pero no el color de forma directa.

Para resolver esta limitación, se emplea habitualmente un filtro de color dispuesto sobre el sensor, siendo el patrón *Bayer* [10] el más común. Este patrón organiza los filtros de color rojo (R), verde (G) y azul (B) en una disposición periódica, asignando a cada píxel la captura de un solo componente de color. Dado que el ojo humano es más sensible a la luz verde, el patrón Bayer incluye el doble de filtros verdes en comparación con los rojos y azules. La Figura 6 muestra los cuatro patrones Bayer más comunes utilizados en sensores de imagen digitales.

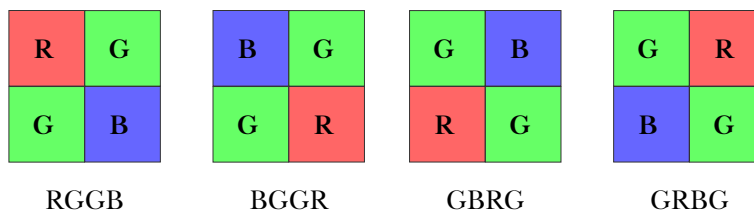


Figura 6: Patrones Bayer más comunes utilizados en sensores de imagen digitales

A partir de esta estructura, la reconstrucción de una imagen a color completa requiere un proceso denominado *demosaicing* o *interpolación de color* [11]. Este proceso tiene como objetivo estimar, para cada píxel, los valores de los componentes cromáticos no capturados directamente por el sensor, a partir de la información proporcionada por los píxeles vecinos y del patrón de filtros utilizado. Como resultado, se obtiene una imagen RGB completa, en la que cada píxel dispone de información en los tres canales de color.

Existen múltiples algoritmos de *demosaicing*, que difieren en complejidad, coste computacional y calidad visual del resultado. Métodos sencillos basados en interpolación bilineal o enfoques más complejos que tienen en cuenta gradientes, bordes y características locales de la imagen. No obstante, todos ellos comparten el mismo objetivo: reconstruir una representación cromática coherente a partir de datos incompletos.

El *demosaicing* constituye el primer paso de cualquier *pipeline de procesamiento de imagen* basado en datos RAW. Un *pipeline* de procesamiento es la secuencia ordenada de etapas que transforman progresivamente los datos capturados por el sensor en una imagen final lista para su visualización o análisis. Estas etapas suelen incluir, además del *demosaicing*, operaciones relacionadas con la corrección y ajuste de la imagen, así como la conversión a un formato de salida estándar para su almacenamiento o visualización. Dicho *pipeline* es la base del Procesamiento Digital de Imágenes.

3.5 Fundamentos del Procesamiento Digital de Imágenes

El Procesamiento Digital de Imágenes (PDI) engloba el conjunto de técnicas y métodos matemáticos destinados a la manipulación y el análisis de imágenes digitales mediante sistemas de computación. Su objetivo principal es la transformación de una imagen de entrada en otra que resulte más adecuada para una determinada tarea, como la mejora visual, la extracción de información relevante o la preparación para etapas posteriores de procesado [12]. En el contexto de este TFM, el procesamiento digital de imágenes constituye la base sobre la que se aplican los distintos algoritmos aplicados a los datos crudos capturados por el sensor.

3.5.1 Operaciones puntuales y operaciones espaciales

Los métodos matemáticos y algoritmos aplicables sobre imágenes digitales pueden clasificarse como *operaciones puntuales* y *operaciones espaciales*, en función de la información utilizada para calcular el valor de cada píxel en la imagen resultante.

3.5.1.1 Operaciones puntuales

Las operaciones puntuales son aquellas en las que el valor de cada píxel de salida depende únicamente del valor del píxel correspondiente en la imagen de entrada. Este tipo de transformaciones se expresan como (7):

$$I_{salida}(x, y) = T(I_{entrada}(x, y)) \quad (7)$$

donde T es una función de transformación aplicada a cada píxel.

Este tipo de transformaciones no modifica la estructura espacial de la imagen, pero sí su distribución de intensidades, lo que las hace especialmente adecuadas para tareas de ajuste de contraste, corrección de iluminación o normalización de niveles de gris. En el marco de este TFM, las operaciones puntuales más relevantes incluyen:

- Corrección gamma
- Transformaciones sigmoides
- Ecualización del histograma

Estas técnicas presentan un bajo coste computacional y son fácilmente paralelizables, por lo que son muy adecuadas para su implementación eficiente en arquitecturas modernas.

3.5.1.2 Operaciones espaciales

Las operaciones espaciales, en contraposición, son aquellas en las que el valor de cada píxel de salida depende no sólo del píxel de entrada correspondiente, sino de un conjunto de píxeles vecinos. De forma general, pueden expresarse como (8):

$$I_{salida}(x, y) = T(I_{entrada}(S_{x,y})) \quad (8)$$

donde $S_{x,y}$ es el conjunto de píxeles vecinos alrededor de la posición (x, y) y T es una función que combina estos valores.

Este tipo de operaciones permite explotar la correlación espacial existente entre píxeles adyacentes. Son las empleadas en tareas de reducción de ruido, el realce de bordes y la detección de características. En el contexto de este TFM, se consideran principalmente:

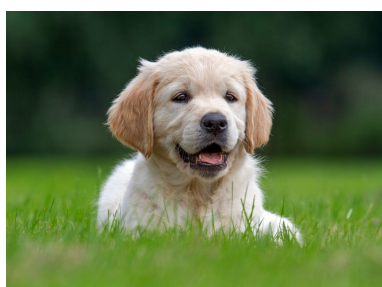
- Filtros lineales
- Filtros no lineales
- *Unsharp Masking*

Estas técnicas suelen implicar un mayor coste computacional debido a la necesidad de acceso a múltiples píxeles para calcular cada valor de salida.

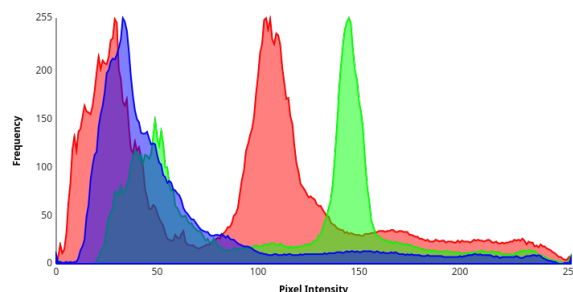
La aplicación de las operaciones anteriormente mencionadas implica generalmente un ajuste de parámetros específicos, que varían en función de las características de la imagen y los objetivos del procesado. Para guiar este ajuste, es necesario utilizar herramientas que permitan evaluar la distribución de intensidades y la calidad visual de la imagen.

3.5.2 El histograma como herramienta de diagnóstico

La representación gráfica del histograma de una imagen digital es la principal herramienta para analizar la distribución de intensidades y evaluar la calidad visual de la misma. Desde el punto de vista matemático, el histograma describe la distribución de frecuencias de los valores de intensidad presentes en la imagen, proporcionando una representación global de cómo se reparte la información luminosa a lo largo de su rango dinámico. En la Figura 7 se muestra un ejemplo de imagen y su histograma RGB correspondiente.



(a) Imagen



(b) Histograma

Figura 7: Imagen y su correspondiente histograma

Interpretar correctamente el histograma permite identificar el correcto revelado de la imagen, así como los ajustes que pueden llegar a ser necesarios para mejorar su apariencia visual [13]. Por ejemplo, un histograma concentrado en la zona izquierda indica una imagen subexpuesta, mientras que una acumulación de valores en la zona derecha sugiere sobreexposición. Asimismo, la presencia de picos pronunciados en los extremos puede evidenciar la pérdida de detalles en sombras o luces debido a saturación.

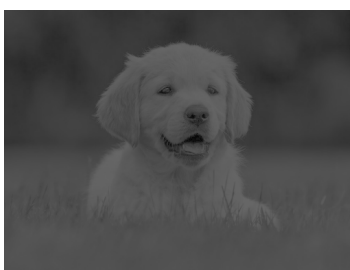
En imágenes con una distribución equilibrada, el histograma suele extenderse a lo largo de una proporción significativa del rango dinámico, reflejando un buen aprovechamiento de los niveles de intensidad disponibles. Sin embargo, una distribución uniforme no implica necesariamente una imagen visualmente correcta, ya que la interpretación del histograma debe realizarse siempre en conjunto con el contenido de la escena y los objetivos del procesado.

3.5.3 Mejora del contraste

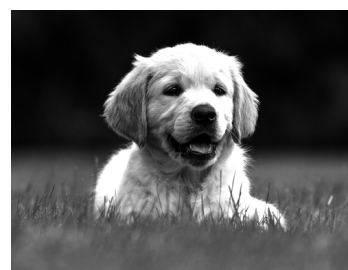
La mejora del contraste es una de las técnicas más comunes en el procesamiento digital de imágenes. Influye directamente en la percepción visual de los detalles y en la calidad global de la imagen. Muchas imágenes, especialmente aquellas capturadas en condiciones de iluminación no controladas o a partir de datos crudos, presentan una distribución de intensidades limitada que no aprovecha el completamnete el rango dinámico disponible. En estos casos, la mejora del contraste permite redistribuir los niveles de intensidad para resaltar detalles relevantes sin introducir artefactos visuales [14].

3.5.3.1 Concepto de contraste

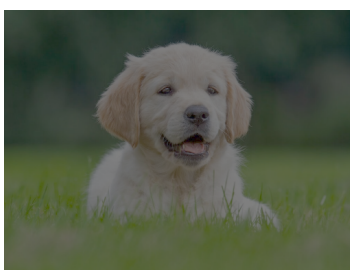
El contraste en una imagen digital se refiere a la diferencia en la intensidad luminosa entre un punto de una imagen y sus alrededores [15]. Un alto contraste implica una diferencia significativa en la luminosidad, lo que facilita la distinción de detalles y características dentro de la imagen. Por el contrario, un bajo contraste resulta en una imagen apagada o deslavada, donde los detalles pueden perderse debido a la falta de variación tonal. Esta diferencia puede observarse en la Figura 8, donde se presentan ejemplos de imágenes con diferentes niveles de contraste, tanto en escala de grises como en color.



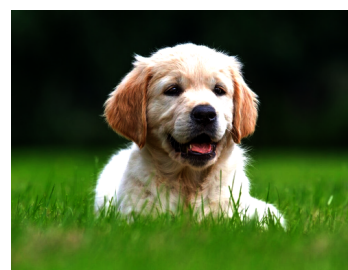
(a) Bajo contraste (gris)



(b) Alto contraste (gris)



(c) Bajo contraste (color)



(d) Alto contraste (color)

Figura 8: Ejemplos de imágenes con diferentes niveles de contraste

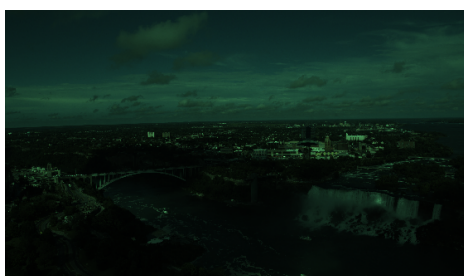
A continuación, se describen una serie de técnicas de mejora del contraste comúnmente empleadas en el procesamiento digital de imágenes y que son relevantes para el desarrollo de este TFM.

3.5.3.2 Ecualización global del histograma

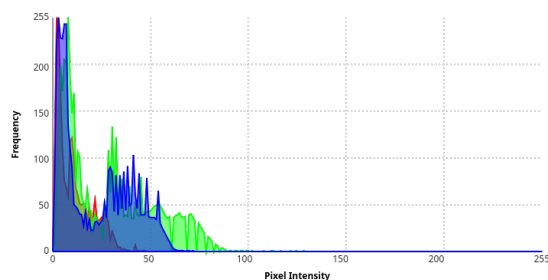
La ecualización global del histograma [16] es una técnica clásica de mejora del contraste que actúa sobre la imagen completa. Su objetivo es redistribuir los valores de intensidad de forma que el histograma resultante sea aproximadamente uniforme a lo largo del rango dinámico disponible.

Esta técnica se basa en una transformación no lineal de los valores de intensidad, calculada a partir de la función de distribución acumulada del histograma original. Como resultado, regiones con bajo contraste inicial pueden volverse más distinguibles. Sin embargo, al tratarse de una operación global, la ecualización del histograma puede producir efectos no deseados, como la amplificación del ruido o la pérdida de naturalidad en ciertas escenas, especialmente cuando existen grandes variaciones locales de iluminación.

En la Figura 9 se muestra un ejemplo de aplicación de la ecualización global del histograma sobre una imagen. La imagen original (Figura 9a) es una imagen en formato crudo en etapas tempranas de revelado. Su histograma correspondiente (Figura 9b) muestra una distribución de intensidades concentrada en la parte inferior del rango dinámico. Tras aplicar la ecualización del histograma, se obtiene la imagen ecualizada (Figura 9c), que presenta un contraste significativamente mejorado. El histograma resultante (Figura 9d) muestra una distribución más uniforme de los niveles de intensidad. En este caso, la imagen resultante exhibe un detalle visual muy mejorado.



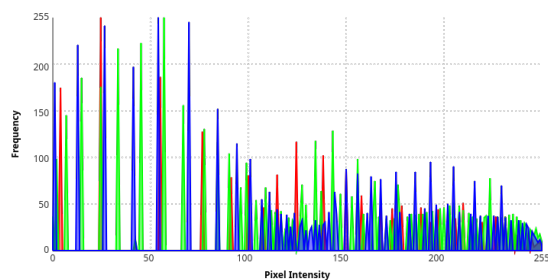
(a) Imagen original



(b) Histograma original



(c) Imagen resultante



(d) Histograma resultante

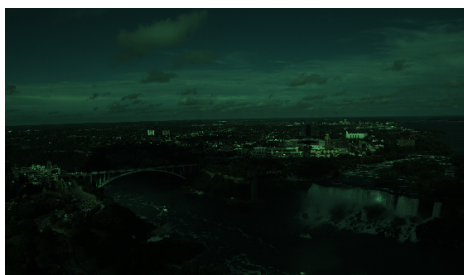
Figura 9: Ejemplo de aplicación de ecualización global del histograma

3.5.3.3 Ecualización adaptativa y CLAHE

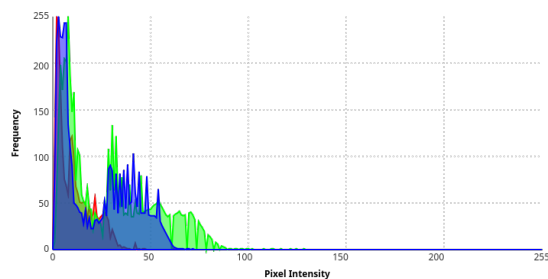
La ecualización global del histograma no considera las variaciones locales de contraste dentro de la imagen, sino que aplica una transformación uniforme a toda la imagen. Para superar esta limitación, se emplean técnicas de ecualización adaptativa [17], que actúan sobre regiones locales de la imagen en lugar de hacerlo a nivel global. Siguiendo este enfoque, la imagen se divide en múltiples regiones, y se aplica la ecualización del histograma de forma independiente en cada una de ellas. Así se consigue una mejora del contraste a nivel local, sin aplicar generalizaciones que puedan resultar inapropiadas para ciertas áreas de la imagen.

Una variante ampliamente utilizada es la ecualización adaptativa con limitación de contraste (*Contrast Limited Adaptive Histogram Equalization*, CLAHE) [18]. Este método introduce un mecanismo de control que evita la amplificación excesiva del contraste y del ruido, limitando el crecimiento de los picos del histograma local.

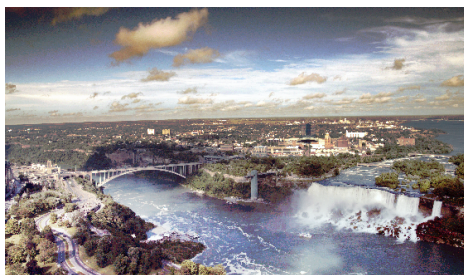
En la Figura 10 se muestra un ejemplo de aplicación de CLAHE sobre una imagen. De nuevo, la imagen original (Figura 10a) es una imagen en formato crudo en etapas tempranas de revelado. El resultado en este caso, muestra una mejora del contraste más sutil pero efectiva (Figura 10c), con un histograma resultante (Figura 10d) que refleja una distribución de intensidades mejorada sin los picos tan pronunciados de la ecualización global (Figura 9d).



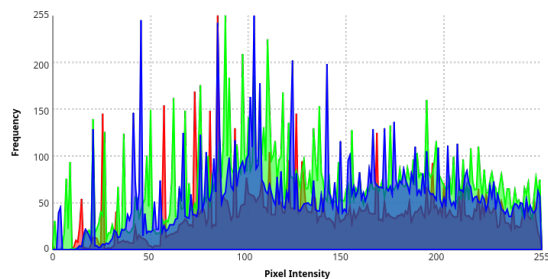
(a) Imagen original



(b) Histograma original



(c) Imagen resultante



(d) Histograma resultante

Figura 10: Ejemplo de aplicación de CLAHE

3.5.3.4 Normalización

La normalización es una operación destinada a reescalar los valores de intensidad de una imagen para que se ajusten a un rango determinado. Su objetivo principal es mejorar el aprovechamiento del rango dinámico disponible y facilitar la comparación entre imágenes o su posterior procesado.

3.5.3.4.1 Normalización min-max

La normalización min-max [19] es una de las técnicas más sencillas y utilizadas para el reescalado de intensidades. Este método consiste en mapear el valor mínimo presente en la imagen al extremo inferior del rango de destino, y el valor máximo al extremo superior. De este modo, los valores intermedios se redistribuyen de forma proporcional, permitiendo que la imagen utilice la totalidad del rango dinámico disponible. Matemáticamente, la normalización min-max se expresa como (9):

$$I_{salida}(x, y) = \frac{I_{entrada}(x, y) - I_{min}}{I_{max} - I_{min}} \quad (9)$$

Es importante mencionar que, en imágenes con más de un canal (por ejemplo, imágenes en color RGB), la normalización se aplica de forma independiente a cada canal para preservar la relación cromática original. La normalización resulta muy útil cuando la imagen presenta un contraste reducido debido a una ocupación limitada del rango de intensidades, aunque puede verse afectada por la presencia de valores extremos atípicos que condicionen el reescalado global.

En la Figura 11 se muestra un ejemplo de aplicación de la normalización min-max sobre una imagen a color.



(a) Imagen original



(b) Imagen normalizada

Figura 11: Ejemplo de aplicación de normalización min-max

3.5.3.4.2 Normalización min-max con percentiles

Para mitigar la influencia de valores extremos, es habitual emplear una variante de la normalización min-max basada en percentiles. En este caso, en lugar de utilizar los valores mínimo y máximo absolutos de la imagen, se seleccionan percentiles bajos y altos del histograma como límites del rango de reescalado. Matemáticamente, esta técnica se define como (10):

$$I_{salida}(x, y) = \frac{I_{entrada}(x, y) - P_{inf}}{P_{sup} - P_{inf}} \quad (10)$$

P_{inf} y P_{sup} representan los percentiles inferior y superior seleccionados, respectivamente. La Figura 12 muestra un ejemplo de aplicación de la normalización min-max con percentiles sobre una imagen a color, utilizando los percentiles 2 y 98 como límites de reescalado.



(a) Imagen original



(b) Imagen normalizada

Figura 12: Ejemplo de aplicación de normalización min-max con percentiles

3.5.3.5 Transformaciones no lineales de intensidad

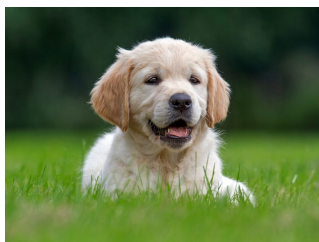
Además de las técnicas basadas en la redistribución del histograma y normalización, la mejora del contraste puede aplicarse realizando transformaciones no lineales sobre los valores de intensidad. Estas transformaciones modifican la relación entre los valores de entrada y salida de forma controlada, permitiendo ajustar selectivamente determinadas regiones del rango dinámico sin redistribuir explícitamente el histograma.

3.5.3.5.1 Corrección gamma

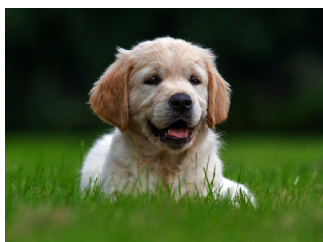
La corrección gamma es una transformación no lineal que ajusta la luminosidad de una imagen. Consiste en la aplicación de una función exponencial a los valores de intensidad, controlada por un parámetro denominado *gamma* (γ) [20]. Se define mediante la siguiente ecuación (11):

$$I_{salida}(x, y) = I_{entrada}(x, y) \cdot c^{\gamma} \quad (11)$$

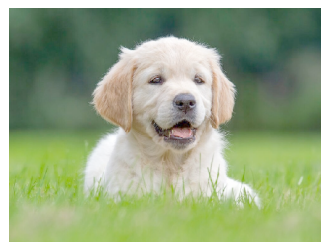
Valores de $\gamma < 1$ realzan las zonas oscuras de la imagen, mientras que valores de $\gamma > 1$ atenúan las zonas más claras. La constante c se utiliza para normalizar los valores de intensidad y asegurar que permanezcan dentro del rango válido. La Figura 13 muestra un ejemplo de aplicación de la corrección gamma sobre una imagen a color.



(a) Imagen original



(b) Con $\gamma = 1,5$ y $c = 1$



(c) Con $\gamma = 0,5$ y $c = 1$

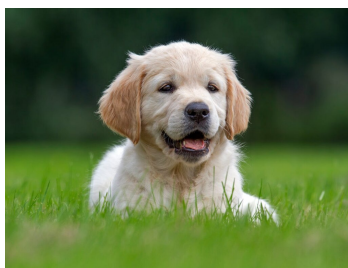
Figura 13: Ejemplo de corrección gamma

3.5.3.5.2 Función sigmoide

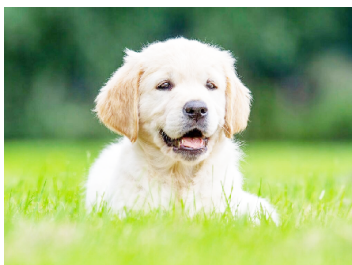
La mejora del contraste también puede realizarse mediante la aplicación de la función sigmoide, que proporciona una transición suave entre las regiones de baja y alta intensidad. A diferencia de la corrección gamma, la función sigmoide permite enfatizar el contraste en torno a un rango de intensidades específico, controlando la pendiente y el punto de inflexión de la curva. Matemáticamente, se expresa como (12):

$$I_{salida}(x, y) = \frac{1}{1 + e^{-k(I_{entrada}(x,y) - I_0)}} \quad (12)$$

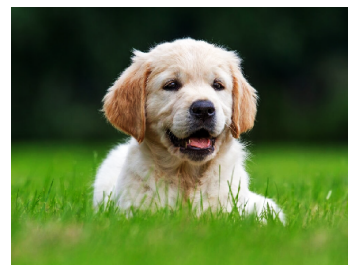
donde k , también denominada *gain* (ganancia), controla la pendiente de la curva y I_0 determina el punto de inflexión (*cutoff*). Normalmente, suele aplicarse una normalización posterior para ajustar los valores de salida al rango dinámico válido. La Figura 14 muestra un ejemplo de mejora del contraste mediante la función sigmoide sobre una imagen a color.



(a) Imagen original



(b) Con $k = 5$ y $I_0 = 0$



(c) Con $k = 5$ y $I_0 = 0,5$

Figura 14: Ejemplo de mejora del contraste mediante función sigmoide

3.5.4 Filtrado espacial y reducción de ruido

Las operaciones de mejora del contraste suelen ir acompañadas de técnicas de filtrado espacial. Esto suele ser muy común cuando la imagen presenta ruido o irregularidades introducidas durante la captura o el proceso de digitalización. El ruido se manifiesta habitualmente como variaciones aleatorias de intensidad que pueden degradar la calidad visual y dificultar la interpretación de los detalles presentes en la imagen.

El filtrado espacial [21] consiste en la modificación del valor de cada píxel teniendo en cuenta los valores de sus píxeles vecinos, con el objetivo de suavizar la imagen, reducir el ruido o resaltar características específicas. En función de la forma en la que se combinan los valores locales, los filtros espaciales pueden clasificarse en lineales y en no lineales. A continuación, se describen dos de los filtros más representativos de cada categoría, que son relevantes para el desarrollo de este TFM.

3.5.4.1 Filtrado lineal: filtro gaussiano

El filtro gaussiano [22] es uno de los métodos más empleados para la reducción de ruido de alta frecuencia. Se trata de un filtro lineal que realiza un suavizado progresivo de la imagen mediante la convolución con una máscara basada en la función gaussiana. La función gaussiana en dos dimensiones se define como (13):

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (13)$$

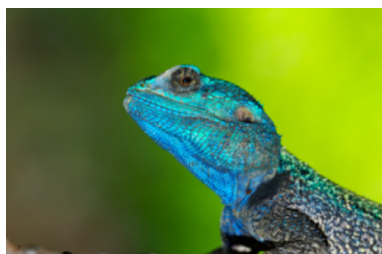
donde σ es la desviación estándar que controla el grado de suavizado. Además, la máscara del filtro gaussiano se realiza sobre una ventana de tamaño $k \times k$, donde k es un número impar que determina el área de influencia del filtro. Computacionalmente, esta ventana suele denominarse *kernel*.

Este tipo de filtrado atenúa variaciones abruptas de intensidad, lo que contribuye a eliminar ruido aleatorio y pequeñas irregularidades. Sin embargo, como efecto secundario, el filtro gaussiano también puede provocar una ligera pérdida de nitidez y difuminar los bordes de la imagen.

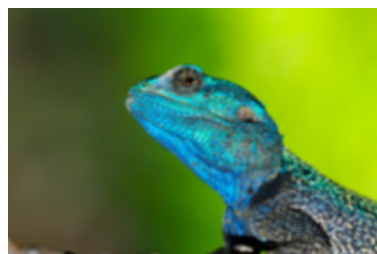
En la Figura 15 se muestra un ejemplo de aplicación del filtro gaussiano sobre una imagen a color, utilizando diferentes valores de k y σ , que ilustran el efecto de suavizado progresivo.



(a) Imagen original



(b) Con $k = 7$ y $\sigma = 5$



(c) Con $k = 13$ y $\sigma = 9$

Figura 15: Ejemplo de aplicación de filtro gaussiano

3.5.4.2 Filtrado no lineal: filtro de mediana

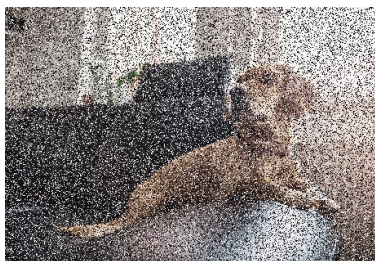
El filtro de mediana [23] es un método de filtrado no lineal diseñado específicamente para la eliminación del ruido impulsivo, como el conocido ruido de tipo *sal y pimienta* [24]. A diferencia de los filtros lineales, este filtro sustituye directamente el valor de cada píxel por la mediana de los valores de intensidad presentes en su vecindad definida por una ventana de tamaño $k \times k$. Matemáticamente, el proceso se describe como (14):

$$I_{salida}(x, y) = \text{mediana}\{I_{entrada}(i, j) \mid (i, j) \in S_{x,y}\} \quad (14)$$

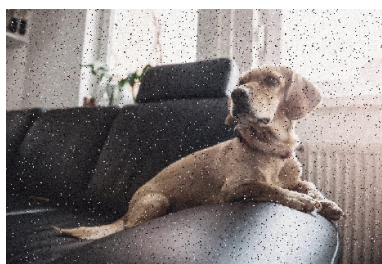
donde $S_{x,y}$ es el conjunto de píxeles vecinos alrededor de la posición (x, y) .

El filtro de mediana es especialmente eficaz para eliminar píxeles atípicos sin afectar significativamente los bordes y detalles de la imagen. Esto se debe a que la mediana es menos sensible a valores extremos. Otros filtros, como el filtro de media [25] o el mismo filtro gaussiano, suelen difuminar los bordes y detalles al promediar los valores de intensidad. En contrapartida, el filtro de mediana es más costoso computacionalmente.

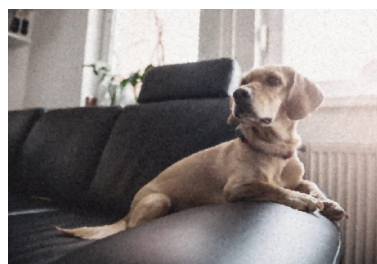
En la Figura 16 se muestra un claro ejemplo de aplicación del filtro de mediana sobre una imagen a color con ruido de tipo sal y pimienta, utilizando diferentes tamaños de ventana k .



(a) Imagen original



(b) Con $k = 3$



(c) Con $k = 7$

Figura 16: Ejemplo de aplicación de filtro de mediana

3.5.5 Realce de detalle y nitidez

Además de la mejora del contraste y la reducción de ruido, es habitual aplicar técnicas destinadas a realzar el detalle y aumentar la percepción de nitidez de la imagen. Estas operaciones no añaden nueva información, sino que actúan sobre la existente para resaltar transiciones de intensidad, bordes y texturas. Esto hace que los contornos y detalles sean más definidos para el observador [26].

3.5.5.1 Realce de bordes: *Unsharp Masking*

La técnica conocida como *Unsharp Masking* (USM) es una de las más extendidas para el realce de bordes y la mejora de la nitidez aparente de una imagen. A pesar de su nombre, su funcionamiento se basa en el principio opuesto: enfatizar las diferencias locales de intensidad para resaltar los contornos.

De forma conceptual, el proceso consiste en generar una versión suavizada de la imagen original y compararla con esta. La diferencia entre ambas resalta las transiciones de intensidad, que se corresponden principalmente con los bordes. Esta información se añade posteriormente a la imagen original, aumentando el contraste local en dichas zonas y, por tanto, la percepción de nitidez. Matemáticamente se puede expresar como (15):

$$I_{salida}(x, y) = I_{entrada}(x, y) + \sigma (I_{entrada}(x, y) - I_{suavizada}(x, y)) \quad (15)$$

donde $I_{suavizada}(x, y)$ es la imagen suavizada obtenida mediante un filtro y σ es un factor de escala que controla la intensidad del realce. El realce se aplica sobre una ventana de tamaño $k \times k$ alrededor de cada píxel.

En la Figura 17 se muestra un ejemplo de aplicación de *Unsharp Masking* sobre una imagen a color, utilizando diferentes valores de k y σ , que ilustran el efecto de realce progresivo de los bordes y detalles y nitidez de la imagen.



(a) Imagen original



(b) Con $k = 5$ y $\sigma = 3$



(c) Con $k = 13$ y $\sigma = 5$

Figura 17: Ejemplo de aplicación de *Unsharp Masking*

3.5.6 Otras operaciones

Además de todas las técnicas de procesamiento de imágenes descritas anteriormente, existen otras operaciones comunes que suelen ser necesarias en el flujo de trabajo de revelado digital.

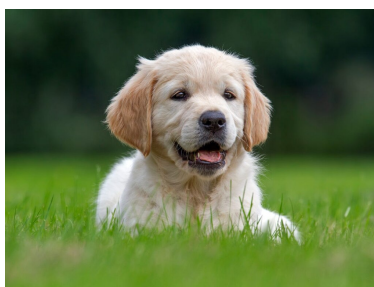
3.5.6.1 Operaciones geométricas

Las operaciones geométricas actúan sobre la estructura espacial de la imagen, alterando la posición de los píxeles sin modificar sus valores de intensidad o color. Su objetivo es el ajuste de la orientación, la disposición o el encuadre de la imagen, manteniendo intacta la información visual contenida en cada píxel.

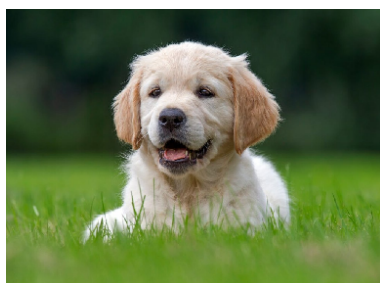
3.5.6.1.1 Volteo

El volteo consiste en la inversión de la imagen respecto a uno de sus ejes principales, ya sea horizontal o vertical. Esta operación se utiliza habitualmente para corregir orientaciones incorrectas derivadas del proceso de captura o para generar imágenes simétricas con fines de análisis o presentación. Desde el punto de vista visual, el contenido de la imagen se conserva, cambiando únicamente su disposición espacial.

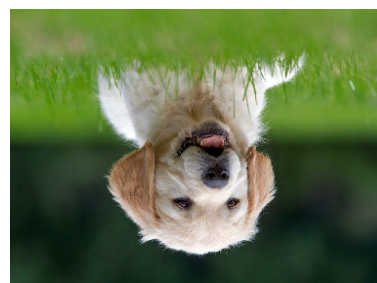
En la Figura 18 se muestra un ejemplo de aplicación de volteo sobre una imagen a color, ilustrando tanto el volteo sobre el eje horizontal como sobre el eje vertical.



(a) Imagen original



(b) Volteo horizontal



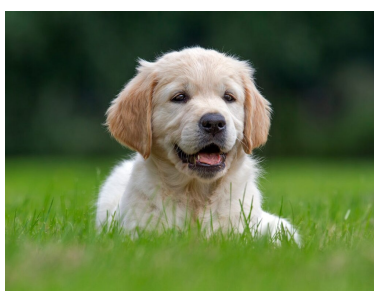
(c) Volteo vertical

Figura 18: Ejemplo de aplicación de volteo

3.5.6.1.2 Rotación

La rotación permite girar la imagen un determinado ángulo alrededor de su centro o de un punto específico. Es una operación común en el ajuste de la orientación de imágenes capturadas con inclinaciones no deseadas o en la normalización de conjuntos de datos visuales. Dependiendo del ángulo de rotación, esta transformación puede requerir un proceso de interpolación para estimar los valores de los nuevos píxeles, lo que puede afectar ligeramente a la calidad final de la imagen.

En la Figura 19 se muestra un ejemplo de aplicación de rotación sobre una imagen a color, ilustrando los resultados obtenidos al rotar la imagen 45° y 135° respectivamente.



(a) Imagen original



(b) Rotación 45°



(c) Rotación 135°

Figura 19: Ejemplo de aplicación de rotación

3.5.6.2 Conversión de espacios de color

La conversión entre espacios de color es una operación habitual en el procesamiento de imágenes. Permite adaptar la representación de la información visual a los objetivos concretos de cada etapa del procesado.

3.5.6.2.1 De RGB a escala de grises

La conversión de una imagen desde el espacio de color RGB a escala de grises consiste en la eliminación de la información cromática, conservando únicamente la información de intensidad luminosa. Esta operación reduce la imagen a un único canal, lo que simplifica el procesamiento posterior.

La conversión no se realiza como una simple combinación de los valores de los canales rojo, verde y azul, sino que se emplean ponderaciones específicas para reflejar la percepción humana del brillo. La fórmula comúnmente utilizada es (16):

$$I_{gris}(x, y) = 0,2989 \cdot I_R(x, y) + 0,5870 \cdot I_G(x, y) + 0,1140 \cdot I_B(x, y) \quad (16)$$

3.5.6.2.2 De RGB a HSV

La conversión de una imagen desde el espacio de color RGB al espacio HSV permite separar la información de color de la información de intensidad. Esta conversión resulta especialmente útil en operaciones que requieren actuar de forma separada sobre el brillo o sobre el color de la imagen, como ajustes de iluminación, segmentación por color o realce selectivo.

Matemáticamente, la conversión [27] se realiza mediante una serie de transformaciones que calculan los componentes de matiz (H), saturación (S) y valor (V) a partir de los valores de los canales rojo, verde y azul. La fórmula completa es más compleja que la conversión a escala de grises, pero se basa en la identificación del valor máximo y mínimo entre los canales RGB y en el cálculo de las diferencias relativas.

3.5.6.3 Conversión de formato de representación para visualización

A lo largo del procesamiento de una imagen es habitual trabajar con valores en coma flotante, ya que este tipo de representación ofrece mayor flexibilidad y precisión durante la aplicación de las distintas operaciones. Sin embargo, para la visualización y el almacenamiento de la imagen en formatos convencionales, es necesaria la conversión de estos valores a una representación entera dentro de un rango finito.

Este proceso consiste en el ajuste de los valores obtenidos al rango admitido por los dispositivos de visualización, normalmente de 8 bits por canal, y la realización de la correspondiente cuantificación. Esta etapa final permite obtener una imagen compatible con los formatos estándar, asegurando que el resultado del procesado pueda ser correctamente interpretado desde el punto de vista visual.

De este modo, la conversión de formato actúa como el paso final que conecta las operaciones de procesamiento con la representación visual de la imagen, cerrando el conjunto de transformaciones descritas en este capítulo.

3.5.7 El *pipeline* de revelado digital

Las operaciones descritas anteriormente no se aplican de forma aislada, sino que forman parte de una secuencia ordenada de etapas conocida como *pipeline* de revelado digital [28]. Este *pipeline* define el orden y la combinación de las distintas transformaciones necesarias para convertir los datos capturados por el sensor en una imagen final visualmente coherente y adecuada para su análisis y visualización.

El *pipeline* de revelado digital comienza con el tratamiento de los datos brutos procedentes del sensor y finaliza con la obtención de una imagen en un formato estándar. Entre ambos extremos, se encadenan diversas operaciones de procesamiento. El orden en el que se aplican es de vital importancia, ya que influye de manera directa en la calidad del resultado final. De forma general, el *pipeline* incluye las etapas que se ilustran en la Figura 20.

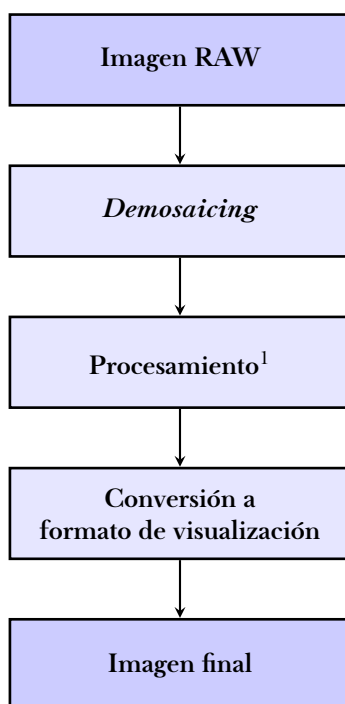


Figura 20: Etapas del *pipeline* de revelado digital

En el marco de este TFM, el *pipeline* de revelado digital es la base para la implementación y optimización de las técnicas de procesamiento. Sobre dicho *pipeline* se desarrollan las pruebas experimentales destinadas a evaluar el rendimiento del sistema mediante la comparativa de diversas configuraciones, dimensiones de imagen y estrategias de ejecución. Asimismo, esta estructura permite la realización de *benchmarks* específicos para analizar el comportamiento computacional, centrándose especialmente en la comparativa de eficiencia entre las implementaciones en CPU y GPU.

¹Incluye mejora del contraste, reducción de ruido, realce de detalle, etc.

3.6 Representación computacional de imágenes digitales

Tras haber analizado la naturaleza de las imágenes digitales, su representación numérica, fundamentos básicos y operaciones de procesamiento empleadas en el revelado digital, es necesario concretar la materialización de estos conceptos en un entorno computacional concreto. La representación computacional de una imagen no es únicamente una cuestión de almacenamiento, sino que condiciona directamente la forma en la que se aplican las operaciones de procesado, la precisión de los cálculos, el consumo de recursos y el rendimiento del sistema.

En los apartados siguientes se describe el modelo de representación adoptado en este TFM, basado en el uso de tensores multidimensionales y apoyado en el *framework* *PyTorch*, que proporciona las herramientas necesarias para la implementación eficiente, flexible y escalable del procesamiento de imágenes digitales.

3.6.1 *PyTorch* como *framework* para el procesamiento de imágenes

PyTorch [29] es una biblioteca de cómputo científico diseñada principalmente para el desarrollo de algoritmos de aprendizaje automático, aunque su estructura y diseño la convierten en una herramienta muy versátil para el cálculo numérico general y el procesamiento de datos de alta dimensión. Además, ofrece un modelo de programación basado en tensores y una gestión transparente de la ejecución tanto en CPU como en GPU.

3.6.2 Tensores en la representación de imágenes

En *PyTorch*, las imágenes digitales se representan mediante tensores [30]. Estos pueden entenderse como generalizaciones de vectores y matrices a múltiples dimensiones. Desde un punto de vista matemático, un tensor de orden n puede expresarse como (17):

$$T_{i_1 i_2 \dots i_n} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n} \quad (17)$$

donde cada índice i_k recorre una dimensión d_k del espacio de datos. Así, un escalar puede considerarse un tensor de orden 0, un vector un tensor de orden 1 y una matriz un tensor de orden 2. Esta generalización permite extender las operaciones algebraicas clásicas a dominios de mayor complejidad, conservando propiedades como la linealidad y la composicionalidad.

De forma simplificada, la Figura 21 ilustra la representación de tensores de diferentes órdenes, ejemplificando cómo se almacenan los datos en cada caso.

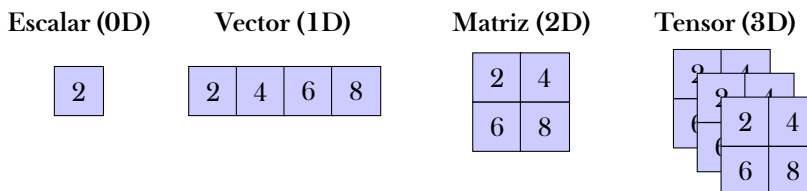


Figura 21: Representación visual de tensores de diferentes dimensiones

Esta disposición permite la representación de imágenes tal y como se describió en Subsección 3.2, donde una imagen a color se almacenaría como un tensor tridimensional (3D) con las dimensiones correspondientes a alto, ancho y canales de color (RGB).

3.6.3 Organización de datos: canales y dimensiones

En *PyTorch*, una imagen se modela como un tensor tridimensional $I \in \mathbb{R}^{C \times H \times W}$, donde C representa el número de canales de color (por ejemplo, $C = 3$ para imágenes RGB) y H y W son las dimensiones de alto y ancho de la imagen, respectivamente.

Cada elemento $I_{c,h,w}$ del tensor almacena la intensidad del píxel ubicado en la posición (h, w) del canal c . Esta representación facilita la manipulación de imágenes, permitiendo aplicar transformaciones mediante simples operaciones tensoriales. Puede observarse una representación conceptual de una imagen RGB como tensor tridimensional en la Figura 22.

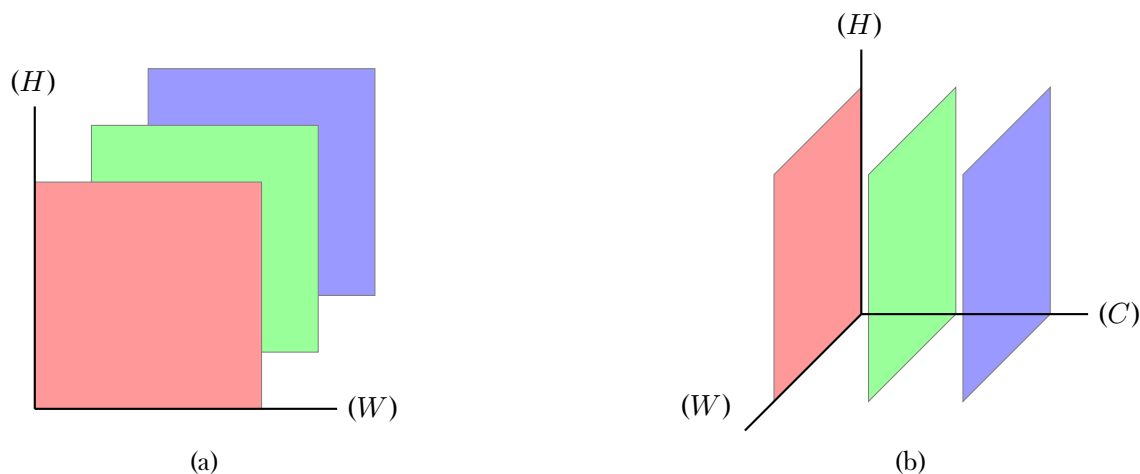


Figura 22: Representación conceptual de una imagen RGB como tensor tridimensional

Numéricamente, una imagen RGB de tamaño 3×3 píxeles puede representarse como el tensor I de forma $[3, 3, 3]$ ², donde cada canal de color contiene una matriz bidimensional con los valores de intensidad correspondientes. En la Figura 23 se muestra un ejemplo concreto de esta representación.

$$I = \left[\begin{matrix} \mathbf{R} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, & \mathbf{G} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, & \mathbf{B} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \end{matrix} \right]$$

Figura 23: Ejemplo de representación tensorial de una imagen RGB de 3×3 píxeles

3.6.4 Tipos de datos y precisión numérica

Los tensores pueden definirse utilizando distintos tipos de datos numéricos, los cuales influyen directamente tanto en la precisión de los cálculos como en el consumo de memoria y el rendimiento computacional. *PyTorch* soporta una amplia variedad de tipos numéricos, que pueden especificarse mediante el parámetro `dtype` [31] durante la creación y manipulación de tensores.

En el contexto del procesamiento de imágenes, los tipos de datos más relevantes son:

- `torch.uint8`: Enteros sin signo de 8 bits, empleados para la representación de imágenes en formatos estándar. Son necesarios para la visualización y el almacenamiento final de las imágenes.
- `torch.float16`: Números en coma flotante de 16 bits, que reducen el consumo de memoria y pueden ofrecer un mayor rendimiento, especialmente en GPU, a costa de una menor precisión.
- `torch.float32`: Números en coma flotante de 32 bits, que proporcionan un equilibrio adecuado entre precisión numérica y eficiencia computacional, siendo el tipo más común durante el procesado.
- `torch.float64`: Números en coma flotante de 64 bits, que ofrecen una mayor precisión numérica, aunque con un mayor coste en memoria y tiempo de cálculo.

A lo largo del *pipeline* de revelado digital, estos tipos de datos se utilizan en diferentes etapas en función de los requisitos de precisión, eficiencia y compatibilidad con las operaciones aplicadas. Asimismo, la elección del `dtype` tiene un impacto directo en el rendimiento del sistema, por lo que su uso será evaluado en los análisis comparativos y *benchmarks* realizados, tanto en CPU como en GPU.

² $[3, 3, 3]$ indica las dimensiones en el orden $[C, H, W]$. Esto es, 3 canales de color, 3 píxeles de alto y 3 píxeles de ancho.

3.7 Procesamiento en tiempo real

En el marco de este TFM, resulta necesario introducir el concepto de *procesamiento en tiempo real*. Este paradigma se refiere a la capacidad de un sistema para procesar datos y producir resultados dentro de unas restricciones temporales bien definidas, de manera que la salida generada sea útil para la aplicación en la que se integra [32]. A diferencia de otros enfoques de procesamiento, los sistemas en tiempo real deben garantizar que cada operación se complete antes de un instante límite o *deadline*.

En el ámbito del procesamiento digital de imágenes, el tiempo real se asocia habitualmente a escenarios en los que las imágenes se capturan y procesan de forma continua, como flujos de vídeo, sistemas de visión artificial, aplicaciones interactivas o procesos de supervisión automática. En estos contextos, el ritmo de procesado viene determinado por la frecuencia de adquisición, de modo que cada imagen debe ser procesada en un intervalo de tiempo suficientemente corto para no introducir retardos acumulativos ni pérdida de datos.

El *pipeline* de revelado digital (Subsubsección 3.5.7) constituye un ejemplo representativo de este tipo de procesamiento. Dicho *pipeline* está formado por una secuencia de etapas, cada una de las cuales introduce un coste computacional específico. Cuando estas etapas se encadenan, el tiempo total de ejecución puede convertirse en un factor limitante si no se diseñan e implementan de manera eficiente.

Además, el tiempo total de ejecución no depende únicamente de la complejidad teórica de los algoritmos empleados, sino también de aspectos prácticos como la organización de los datos, la elección de los tipos numéricos, el paralelismo implícito de las operaciones y el aprovechamiento de los recursos hardware disponibles. En particular, el uso de *PyTorch* permite expresar muchas de estas operaciones de forma vectorizada y facilita su ejecución eficiente tanto en CPU como en GPU.

En este trabajo, el procesamiento en tiempo real se enfoca desde una perspectiva experimental y comparativa. El *pipeline* de revelado digital se utiliza como banco de pruebas para evaluar el rendimiento de diferentes configuraciones, analizando cómo influyen factores como el tamaño de la imagen, la precisión numérica o el dispositivo de ejecución en el tiempo de procesado. Este análisis permitirá estudiar la viabilidad de distintos enfoques bajo restricciones temporales realistas y sentará las bases para la comparación entre implementaciones sobre CPU y GPU en los apartados posteriores.

3.8 Computación acelerada por GPU

La creciente complejidad de los *pipelines* de procesamiento de imágenes y la necesidad de cumplir requisitos de tiempo real hacen que la computación tradicional basada exclusivamente en CPU, resulte, en muchos casos, insuficiente. En este escenario, la computación acelerada por GPU se consolida como una solución imprescindible para la ejecución de tareas intensivas de procesamiento de datos de forma eficiente y escalable.

Las Unidades de Procesamiento Gráfico (GPU) fueron creadas originalmente para acelerar el renderizado gráfico, una tarea que requiere la ejecución de un gran número de operaciones similares de forma simultánea. A diferencia de las CPUs (Unidad Central de Procesamiento), las GPUs adoptan una arquitectura orientada al paralelismo masivo. Esto se traduce en la presencia de un elevado número de núcleos simples, capaces de ejecutar la misma operación sobre diferentes datos en paralelo. En la Figura 24 se muestra una comparación conceptual entre ambas arquitecturas.

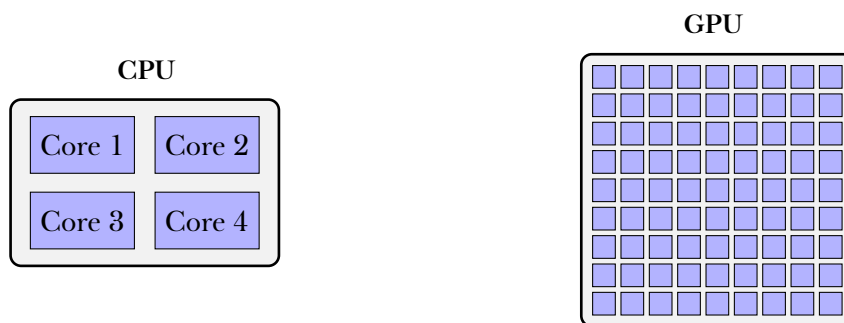


Figura 24: Comparación conceptual entre arquitecturas CPU y GPU

Una CPU cuenta con una cantidad de núcleos o *cores* reducida. Estos núcleos son potentes y están optimizados para manejar tareas secuenciales, con un alto nivel de flujo de ejecución y una baja latencia en la respuesta. Por el contrario, una GPU dispone de cientos o incluso miles de núcleos más simples, organizados para ejecutar una misma instrucción sobre grandes conjuntos de datos. Este modelo de ejecución favorece aquellas tareas en las que se repite la misma operación sobre múltiples elementos independientes, como ocurre en el procesamiento de imágenes y el cálculo matricial. Así pues, cada píxel o pequeño bloque de la imagen puede ser procesado de forma paralela, reduciendo de forma significativa el tiempo total de ejecución.

Dentro del *pipeline* de revelado digital, cada etapa introduce un coste computacional que depende tanto del tamaño de la imagen procesada como del tipo de operación y del formato numérico empleado. A medida que aumenta la resolución de la imagen o la complejidad de las transformaciones aplicadas, el uso exclusivo de CPU puede convertirse en un cuello de botella. La computación acelerada por GPU permite mitigar este problema. Esta coyuntura sirve de fundamento para los análisis y experimentos de *benchmarking* desarrollados en este trabajo.

4 Estudios previos y análisis de alternativas

5 Planificación

6 Presupuesto

7 Análisis

7.1 Requisitos de usuario

7.2 Mapa de historias de usuario

7.3 Prototipos de interfaz de usuario

7.4 Mapa de navegación

8 Diseño

En esta sección se describe la especificación del diseño del sistema, proporcionando una visión global de la arquitectura. Se describen los principales componentes y sus interacciones, con el fin de facilitar la comprensión de la estructura interna y las decisiones de diseño adoptadas.

8.1 Visión general

El sistema desarrollado tiene dos objetivos. Por un lado, proporcionar una interfaz gráfica sencilla, que permita la carga de imágenes en formato crudo y la aplicación de filtros y operaciones de procesamiento. Este proceso tiene como propósito la definición interactiva de *pipelines* de procesamiento de imágenes. Estos *pipelines* deben de poder ser exportados a un formato estándar.

Por otra parte, el sistema debe incorporar un módulo de *benchmarking* que permita analizar el rendimiento computacional de los *pipelines* definidos. Este módulo debe permitir la importación de los *pipelines* creados desde la interfaz gráfica y su ejecución en diferentes condiciones experimentales.

8.1.1 Principios de diseño

Para lograr los objetivos planteados, se han seguido los siguientes principios de diseño:

- **Separación de responsabilidades.** Distinción clara entre componentes encargados de funciones específicas, minimizando las dependencias entre ellos.
- **Modularidad.** Funcionalidades encapsuladas en módulos bien definidos, permitiendo ampliar partes del sistema sin afectar al conjunto.
- **Simplicidad y claridad estructural.** Se evita introducir complejidad innecesaria, manteniendo flujos de trabajo y relaciones entre módulos fácilmente identificables.

8.1.2 Flujo de ejecución

De forma esquemática, el flujo de ejecución del sistema se divide en dos grandes bloques funcionales: la interfaz gráfica y el módulo de *benchmarking*. Se muestra en la Figura 25.

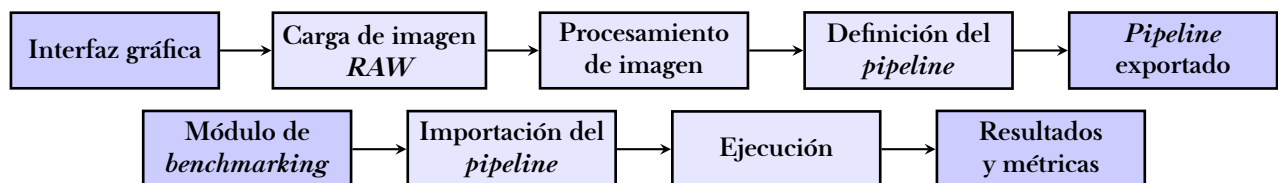


Figura 25: Flujo general de ejecución del sistema

8.2 Arquitectura

Se presenta a continuación la arquitectura del sistema desarrollado. Dicha arquitectura se ha planteado y diseñado siguiendo los objetivos y principios mencionados anteriormente.

8.2.1 Enfoque arquitectónico

El sistema desarrollado sigue un enfoque de arquitectura en capas, combinando el patrón MVC [33] con un modelo de ejecución basado en *pipelines* de procesamiento.

El patrón MVC facilita el desarrollo de la aplicación interactiva, manteniendo desacoplada la interfaz gráfica de la lógica de negocio y de las estructuras de datos y entidades principales. De este modo, las acciones del usuario se traducen en eventos gestionados por un controlador, que coordina la actualización del estado de la aplicación y la representación visual.

Por otra parte, el modelo de ejecución basado en *pipelines* permite definir secuencias de operaciones de procesamiento de imágenes, favoreciendo su reutilización y exportación. Esto facilita la integración con el módulo de *benchmarking*.

La Figura 26 muestra el diagrama general de arquitectura del sistema, donde se identifican los principales componentes y sus interacciones. Cada uno de ellos tiene un rol específico, actuando de forma coordinada para cumplir con los objetivos funcionales.

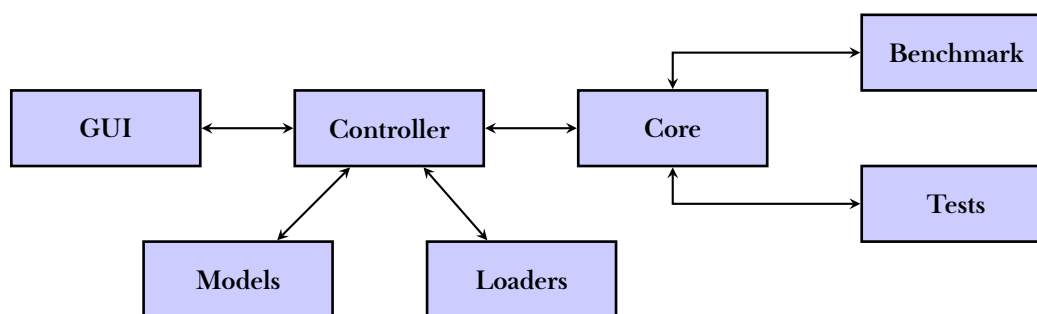


Figura 26: Diagrama de arquitectura del sistema

A partir de esta organización, se logra una arquitectura modular, mantenible y fácilmente extensible, en la que es posible incorporar nuevas operaciones de procesamiento de imágenes, ampliar el soporte de formatos de entrada o introducir nuevas métricas de rendimiento de forma muy sencilla, sin afectar al conjunto global del sistema.

8.3 Componentes

En esta sección, se describen en detalle los componentes principales que conforman el sistema. Se definen sus funciones, responsabilidades y los criterios de diseño específicos que rigen su implementación.

8.3.1 Models

El componente *Models* define las estructuras de datos del sistema, encapsulando la representación de imágenes, operaciones y sus metadatos asociados. Este componente sigue el patrón de diseño *Data Transfer Object* (DTO) y utiliza *dataclasses* [34] de Python para garantizar inmutabilidad y validación de tipos. Se organiza en dos niveles:

- **Modelos.** Clases que representan las entidades centrales del sistema: *Image*, *Metadata*, *OperationDefinition* y *FloatParamSpec*.
- **Enumeraciones.** Tipos enumerados que definen valores válidos para espacios de color, formatos de imagen y patrones Bayer: *ColorSpace*, *ImageFormat* y *Layout*, respectivamente.

A continuación, se describen las clases y enumeraciones de mayor relevancia.

8.3.1.1 Clase *Image*

Constituye el modelo central, representando el estado completo de una imagen durante su procesamiento. Sus atributos principales son:

- **tensor** → Tensor con el estado actual de la imagen.
- **original_tensor** → Tensor inmutable que preserva la imagen original cargada, permitiendo operaciones de reseteo.
- **operation_result_tensor** → Tensor temporal que almacena el resultado de la última operación aplicada, facilitando la previsualización antes de confirmar cambios.
- **path** y **name** → Información de ubicación e identificación del archivo.
- **image_format** → Enumeración que especifica el formato original del archivo.
- **metadata** → Objeto Metadata con información técnica de la imagen.
- **color_space** → Espacio de color actual.

8.3.1.2 Clase *Metadata*

Encapsula los metadatos técnicos asociados a una imagen. Sus atributos incluyen:

- **width y height** → Dimensiones de la imagen en píxeles.
- **bit_depth** → Profundidad de bits.
- **bayer_pattern** → Patrón del filtro de color Bayer, utilizado en la etapa de demosaicing.

8.3.1.3 Clase *OperationDefinition*

Define la especificación completa de una operación de procesamiento de imagen. Actúa como plantilla para todas las operaciones disponibles. Utilizada para encapsular datos y parámetros necesarios para la ejecución de operaciones desde la interfaz de usuario. Sus atributos principales son:

- **name** → Identificador único de la operación.
- **label** → Etiqueta descriptiva para la interfaz de usuario.
- **category** → Categoría de agrupación (“Filters”, “Color”, “Normalization”, “Geometry”, “Format”, “Bayer”).
- **control_type** → Tipo de control GUI asociado (“filter”, “no_param”, “debayer”, “flip”, etc.).
- **params** → Lista opcional de especificaciones de parámetros (FloatParamSpec).
- **default_params** → Diccionario con valores por defecto de los parámetros.

8.3.1.4 Enumeraciones

Proporcionan validación de tipos y documentación explícita de valores permitidos. Se definen las siguientes:

- **ColorSpace** → RGB, HSV y Grayscale.
- **ImageFormat** → RAW, ARW, DNG, PKL_GZ y JPG.
- **Layout** → Patrones Bayer (RGGB, GRBG, GBRG, BGGR), donde cada valor representa los índices de color (R=0, G=1, B=2) en un bloque 2×2.

8.3.2 Loaders

El componente *Loaders* implementa un sistema extensible de carga de imágenes en múltiples formatos. Utiliza un patrón de diseño basado en una clase base abstracta [35] que define la interfaz común para todos los *loaders*. Esto permite añadir soporte para nuevos formatos de imagen sin modificar el código existente. Cada *loader* concreto hereda de esta clase base e implementa la lógica específica para leer y convertir archivos de un formato particular en objetos *Image*. El conjunto de todos los *loaders* disponibles se gestiona mediante un módulo de orquestación que contiene un registro dinámico.

8.3.2.1 Clase base *ImageLoader*

Define la interfaz común para todos los *loaders* de imágenes:

- **extensions** → Propiedad que retorna una secuencia de extensiones soportadas en formato de cadena de texto.
- **formats** → Propiedad que retorna los valores `ImageFormat` correspondientes.
- **load(path, device)** → Método abstracto que carga una imagen desde la ruta especificada y la coloca en el dispositivo PyTorch indicado, retornando un objeto `Image` inicializado.

8.3.2.2 Loaders implementados

Se implementan los siguientes *loaders* concretos, para dar soporte a diferentes formatos de imagen cruda:³

- **RawLoader** → Soporta archivos con extensión `.raw`.
- **ArwLoader** → Soporta archivos con extensión `.arw` (Sony).
- **DngLoader** → Soporta archivos con extensión `.dng` (Adobe, Apple).
- **PklGzLoader** → Soporta archivos con extensión `.pkl.gz` (formato personalizado).

Con el fin de facilitar el desarrollo y pruebas, también se implementa un *loader* (**JpgLoader**) que permite cargar imágenes en formato comprimido `.jpg`.

8.3.2.3 Módulo de orquestación

Gestiona la carga automática de imágenes utilizando el *loader* adecuado según la extensión del archivo. Proporciona una función `load_image(path, device)` que identifica el *loader* correcto y delega la tarea de carga, retornando un objeto `Image`. También ofrece una función `get_supported_formats()` que retorna la lista de todos los formatos de imagen soportados, en función de los *loaders* registrados. Este módulo es utilizado por el *Controller* para gestionar la carga de imágenes desde la interfaz gráfica.

³Existen numerosos formatos de imagen cruda. Información sobre ellos puede encontrarse en el Apéndice C.

8.3.3 Core

Este componente constituye el motor de procesamiento de imágenes. Implementa todas las operaciones de transformación siguiendo, de nuevo, un patrón de diseño basado en una clase abstracta. Las operaciones se organizan en varias categorías temáticas. Cada operación concreta hereda de la clase base e implementa la lógica específica. De esta forma, la implementación de cada operación es completamente autónoma e independiente, además de fácilmente *testable*. También incluye un módulo de utilidades para la gestión de tensores y un sistema de registro dinámico de todas las operaciones disponibles.

8.3.3.1 Clase base *ImageOperation*

Define la interfaz común para todas las operaciones de procesamiento de imágenes:

- **`__call__(x)`** → Punto de entrada público que:
 1. Valida que la entrada sea un tensor 3D (C,H,W).
 2. Invoca el método abstracto `apply()`.
 3. Valida que la salida sea un tensor 3D.
 4. Garantiza que el tensor de salida sea contíguo en memoria.
 5. Proporciona validación de tipos de entrada y salida.
- **`apply(x)`** → Método abstracto que implementa la lógica específica de cada operación.

Además, con fines de análisis, la clase base incluye un parámetro (`execution_time`) que almacena el tiempo de ejecución de la última llamada a la operación, medido en milisegundos. Esta medida se muestra en la interfaz gráfica para proporcionar retroalimentación al usuario sobre el rendimiento de cada operación aplicada.

8.3.3.2 Sistema de registro

Utiliza un diccionario global (`OPERATION_REGISTRY`) que mapea los nombres de las operaciones a sus clases concretas. Además, proporciona un decorador (`@register_operation`) que registra automáticamente cada clase de operación al momento de su definición. Este sistema de registro es utilizado gestionado por el *Controller* y por el módulo de *benchmarking* de forma completamente independiente y dinámica para la invocación de operaciones.

8.3.3.3 Operaciones implementadas

Se implementan todas las operaciones de procesamiento de imágenes descritas previamente en la Subsección 3.5, organizadas en las siguiente categorías:

- **Filters.** Operaciones de filtrado y mejora de contraste.
 - **SigmoidContrast** → Mejora de contraste mediante función sigmoide.
 - **GaussianFilter** → Filtro de desenfoque gaussiano.
 - **MedianFilter** → Filtro de mediana para reducción de ruido.
 - **GammaAdjustment** → Ajuste gamma para corrección tonal.
 - **HistogramEqualization** → Ecualización de histograma.
 - **CLAHE** → Ecualización adaptativa de histograma con limitación de contraste.
 - **UnsharpMask** → Máscara de enfoque para realce de bordes.
- **Color.** Operaciones de conversión entre espacios de color.
 - **ColorToGray** → Conversión RGB a escala de grises.
 - **GrayToColor** → Conversión de escala de grises a RGB.
 - **RgbToHsv** → Conversión de espacio de color RGB a HSV.
 - **HsvToRgb** → Conversión de espacio de color HSV a RGB.
- **Geometry.** Transformaciones geométricas sobre la imagen.
 - **Flip** → Volteo horizontal o vertical.
 - **Rotate** → Rotación de la imagen.
- **Normalization.** Normalización de valores de píxeles.
 - **MinMaxNormalization** → Normalización Min-Max estándar.
 - **MinMaxPercentileNormalization** → Normalización Min-Max con percentiles.
- **Bayer.** *Demosaicing* de imágenes RAW con patrón Bayer.
 - **Debayer** → Conversión de imagen Bayer monocanal a RGB. Se implementan diferentes módulos de *demosaicing* que implican distintos compromisos entre calidad y rendimiento:
 - **Debayer2x2** → Interpolación bilineal básica.
 - **Debayer3x3** → Interpolación con kernel 3x3.
 - **Debayer5x5** → Interpolación con kernel 5x5.
 - **DebayerSplit** → Método alternativo basado en separación de canales.
- **Format.** Conversiones de formato de datos.
 - **RealToRGB8** → Conversión de coma flotante a enteros de 8 bits.
 - **RGB8ToReal** → Conversión de enteros de 8 bits a coma flotante.

8.3.4 Controller

El *Controller* actúa como el coordinador central del sistema. Su responsabilidad principal es la orquestación de la interacción de entre la interfaz gráfica, los *loaders* y el motor de procesamiento (*Core*). Mantiene el estado de la imagen cargada, gestiona el *pipeline* de operaciones y mantiene un registro de eventos. A continuación, se describen en detalle sus principales cometidos.

8.3.4.1 Gestión de la imagen y su estado

Cuando el usuario carga una imagen a través de la interfaz gráfica, el *Controller*

1. La guarda como imagen actual.
2. Controla su estado tras la aplicación de operaciones.
3. Mantiene una copia de referencia de la imagen original para permitir resets.

Este flujo garantiza que el usuario pueda experimentar con diferentes operaciones sin perder la imagen inicial, pero no permite la ejecución secuencial de múltiples operaciones acumulativas. Tampoco permite deshacer operaciones individuales.

8.3.4.2 Gestión del *pipeline* de operaciones

Para mitigar las limitaciones anteriores, el *Controller* mantiene una lista ordenada de operaciones que constituyen el *pipeline* de procesamiento definido por el usuario. Cuando el usuario aplica una nueva operación, esta se añade al final de la lista, recalculando el resultado para toda la secuencia. Esto permite:

1. Aplicar múltiples operaciones de forma acumulativa.
2. Modificar, reordenar o eliminar operaciones individuales.
3. Previsualizar el resultado tras cada cambio en el *pipeline*.

Sin embargo, este planteamiento es computacionalmente costoso para *pipelines* largos o imágenes de gran tamaño, ya que cada modificación requiere recalcular desde el inicio.

8.3.4.3 Sistema de *snapshots*

Para optimizar el rendimiento, el *Controller* implementa un sistema de *snapshots*. Este sistema guarda estados intermedios de la imagen tras la aplicación de cierto número de operaciones. De este modo, cuando se modifica una operación en el *pipeline*, el *Controller* puede restaurar el estado más reciente guardado antes de la operación modificada y continuar desde ese punto, en lugar de recalculer continuamente desde el inicio. En la Figura 27 se ilustra este concepto.

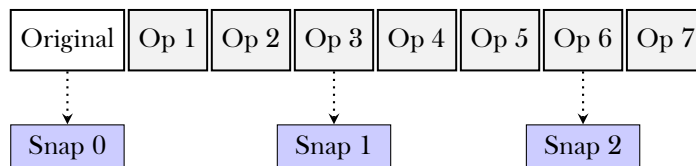


Figura 27: Sistema de *snapshots* para optimización del *pipeline* de operaciones

Se observa como, tras aplicar 3 operaciones, se guarda el primer *snapshot* (Snap 1). Después de aplicar otras 3 operaciones, se guarda el segundo *snapshot* (Snap 2). Si el usuario modifica la operación 5, el *Controller* puede restaurar el estado desde Snap 1 y continuar desde ahí, aplicando las operaciones 5, 6 y 7, en lugar de recalculer desde el estado original.

8.3.4.4 Actualización de la interfaz

Por último, el *Controller* es responsable de manejar las actualizaciones de la interfaz gráfica. Cuando el usuario realiza una acción (carga de imagen, aplicación de operación, modificación del *pipeline*, etc.), el *Controller* procesa la solicitud y notifica a la GUI para que refresque la visualización correspondiente. La interfaz siempre representa el estado real del sistema y el usuario recibe una respuesta inmediata a sus acciones.

Además, el *Controller* mantiene un registro de todas las acciones realizadas por el usuario durante una sesión. Dicho registro se muestra en la interfaz en forma de *logger* para proporcionar retroalimentación y facilitar el seguimiento de cambios realizados.

8.3.5 GUI

El componente *GUI* implementa la interfaz gráfica utilizando PyQt6 [36]. Ocupa varias responsabilidades: la presentación de la imagen, el manejo de la interacción del usuario y la visualización de información relevante sobre el procesamiento. Su diseño sigue una estructura completamente modular, donde cada panel o *widget* tiene una función específica bien definida. En la Figura 28 y la Figura 29 se muestra una visión general.

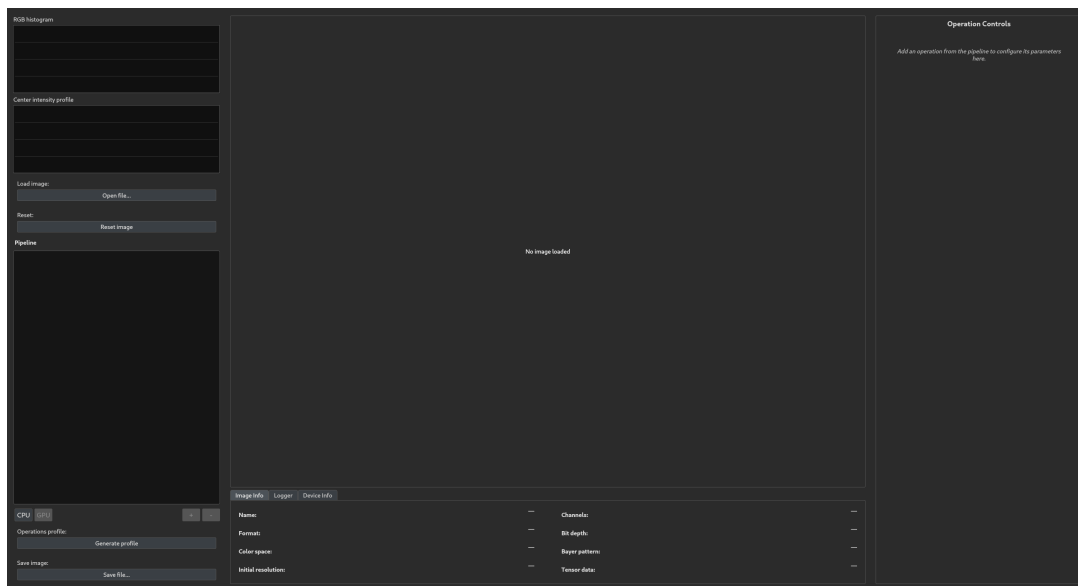


Figura 28: Visión general de la interfaz gráfica (sin imagen cargada)

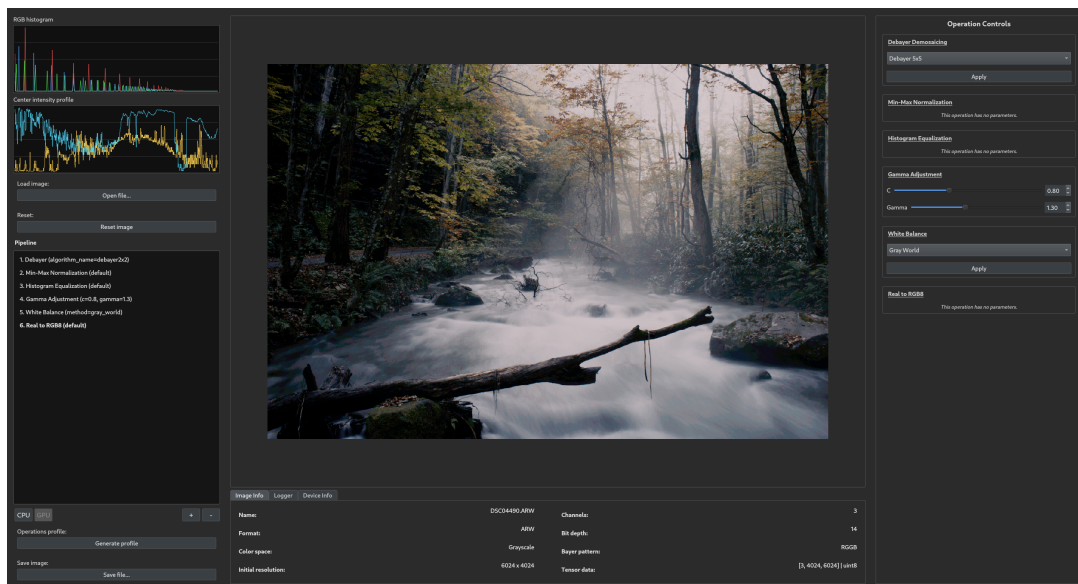


Figura 29: Visión general de la interfaz gráfica (con imagen cargada)

La interfaz se organiza en tres paneles principales distribuidos verticalmente:

- Panel izquierdo (LeftPanel)
- Visor central (ImageViewer)
- Panel derecho (RightPanel)

8.3.5.1 Panel izquierdo (*LeftPanel*)

El panel izquierdo es el encargado de gestionar las principales acciones que se pueden realizar desde la interfaz gráfica. Actúa como centro de control. En la Figura 30 se muestra su disposición respecto al resto de la interfaz.

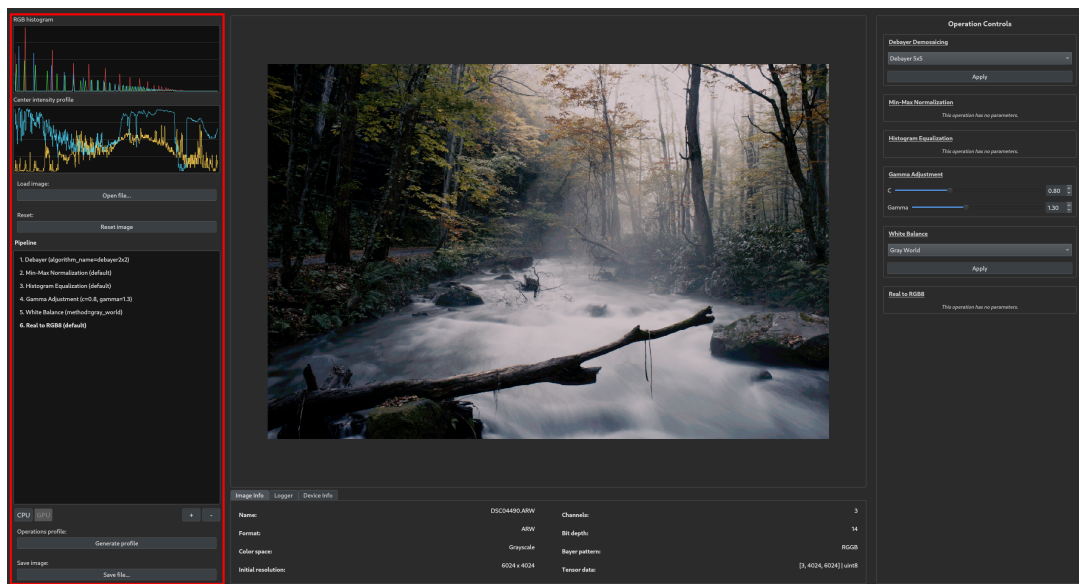


Figura 30: Panel izquierdo (*LeftPanel*)

Los primeros elementos que renderiza este componente son dos gráficos que muestran información de análisis sobre la imagen cargada, los cuales se muestran en la parte superior del panel.

- **Histograma RGB.** Muestra la distribución de valores de píxeles para cada canal de color.
- **Perfiles centrales de intensidad.** Representa la intensidad de los perfiles vertical y horizontal que atraviesan el centro de la imagen.

En la Figura 31 se muestran dichos gráficos.

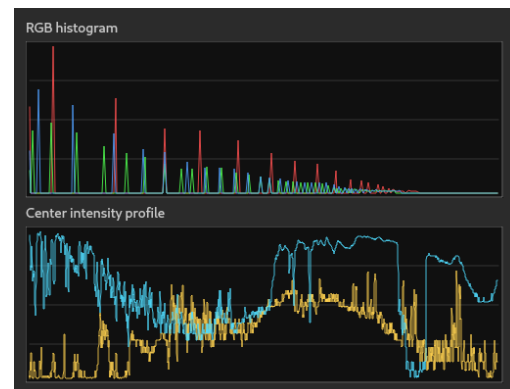


Figura 31: Gráficos de información de la imagen

Justo debajo de los gráficos, se renderizan dos botones. El primero de ellos permite cargar una nueva imagen completamente desde cero, mientras que el segundo restablece la imagen actual a su estado original, eliminando todas las operaciones de procesamiento aplicadas. Dichos botones se muestran en la Figura 32.

Tras estos botones, se renderiza uno de los componentes más importantes de la interfaz, el *selector de operaciones* (Figura 33) para el *pipeline* de procesamiento. Este componente permite añadir y eliminar operaciones a la secuencia de procesamiento, mostrando una lista ordenada de las operaciones aplicadas y sus respectivos parámetros. Actúa como una pila: añade nuevas operaciones al final de la lista y permite eliminar la última operación aplicada. Esta pila se maneja con los botones “+” y “-” ubicados en la parte inferior derecha del componente. El botón “+” despliega un menú con todas las operaciones disponibles, organizadas por categorías. Cada operación se aplica con sus parámetros por defecto, aunque el usuario puede modificarlos posteriormente. Además, en la parte inferior izquierda del componente, se renderiza un botón de tipo *toggle* [CPU/GPU] que permite cambiar el dispositivo de procesamiento en el que se ejecutan las operaciones, mostrando el dispositivo seleccionado.

Finalmente, en la parte inferior del panel izquierdo, se renderizan dos botones adicionales (Figura 34). El primero de ellos genera un perfil de ejecución en formato *json* que representa el *pipeline* de operaciones. Su función es la exportación de la secuencia de operaciones aplicadas para su análisis en el módulo de *benchmarking*. El segundo botón permite guardar la imagen procesada en formato *jpg*.

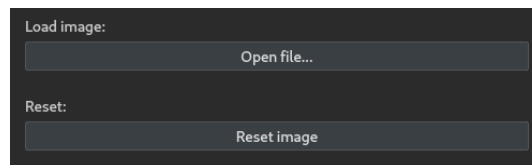


Figura 32: Botones de guardado y reinicio



Figura 33: Selector de operaciones

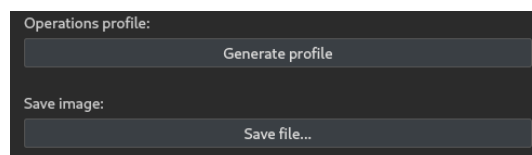


Figura 34: Botones para generar el perfil de ejecución de operaciones y para guardar la imagen procesada

8.3.5.2 Visor central (*ImageViewer*)

El visor central es el componente principal de toda la interfaz. Es el encargado de mostrar la imagen cargada y procesada, actualizándose cada vez que se aplica una nueva operación o se lleva a cabo un ajuste de parámetros. Además, muestra información técnica sobre la imagen y el propio procesamiento. En la Figura 35 se muestra su disposición respecto al resto de la interfaz.

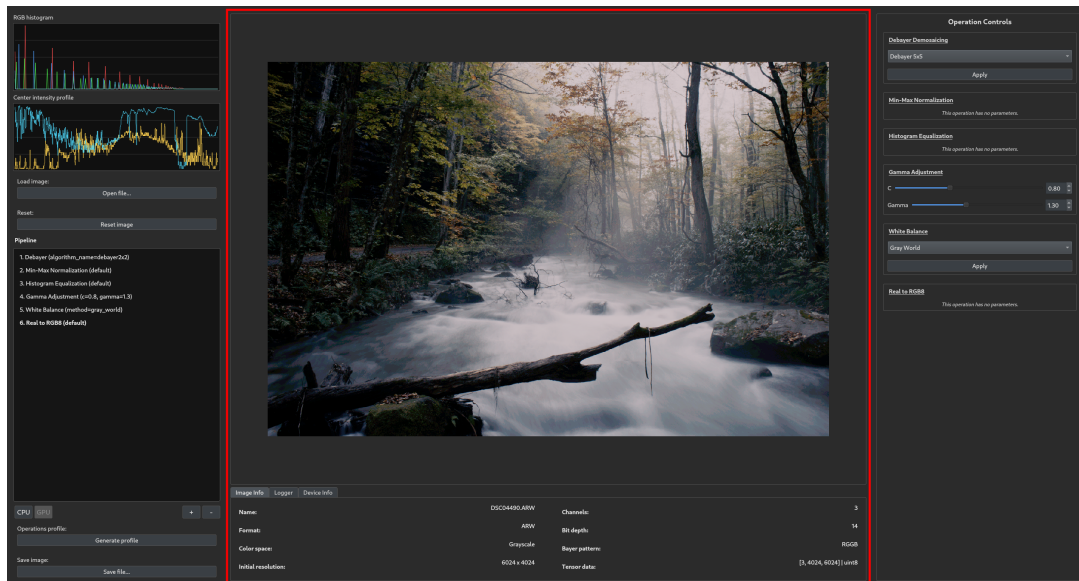


Figura 35: Visor central (*ImageViewer*)

Este componente se divide en dos partes claramente diferenciadas: un visor principal que muestra la imagen y una sección de *tabs* inferior que muestra información. El visor principal permite, además de visualizar la imagen, realizar acciones de zoom y desplazamiento para facilitar la inspección de detalles. La sección de *tabs* se organiza en tres pestañas: *Image Info*, *Logger* y *Device Info*, como se puede observar en la Figura 36.

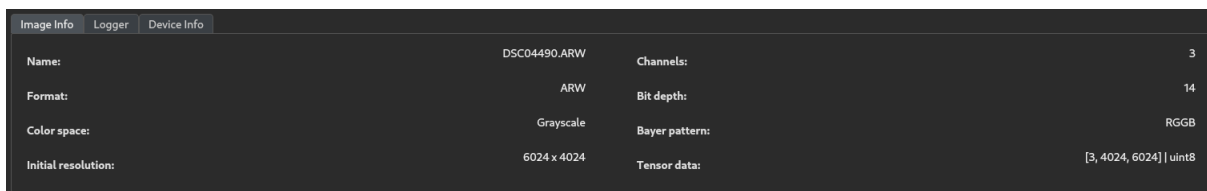


Figura 36: Sección de *tabs* del visor central

La primera de las pestañas (*Image Info*) muestra información técnica sobre la imagen cargada:

- Nombre de la imagen
- Formato
- Espacio de color
- Dimensiones (ancho $\times$ alto)
- Número de canales
- Profundidad de bits
- Patrón Bayer (si aplica)
- Estado del tensor $\rightarrow [C, H, W] \mid$ Tipo de dato

La segunda de las pestañas (*Logger*) muestra un registro de eventos que representa todas las acciones realizadas por el usuario durante la sesión actual. Esta pestaña tiene una función meramente informativa, proporcionando retroalimentación al usuario sobre las operaciones aplicadas, los cambios de parámetros, el dispositivo de procesamiento seleccionado, etc. En la Figura 37 se muestra un ejemplo de esta pestaña, que refleja el registro de acciones realizadas para la toma de las capturas de pantalla anteriores: Carga de imagen → Aplicación de *Debyaer* → Aplicación de Normalización mín. máx. → Aplicación de Ecualización de Histograma → Aplicación de Ajuste *Gamma*.

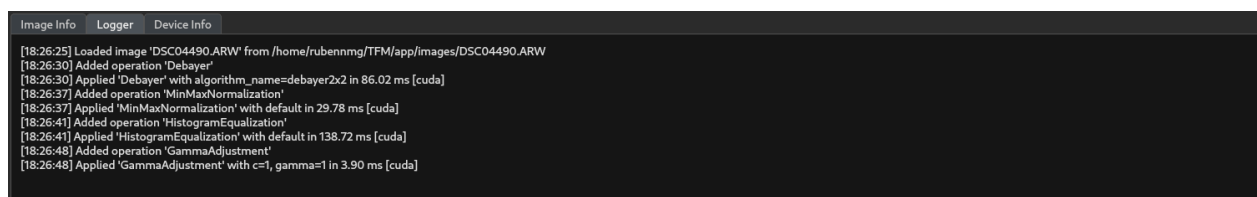


Figura 37: Pestaña de *Logger* del visor central

Además, tras la aplicación de cada operación, se muestra el tiempo de ejecución de la misma en milisegundos, que permite obtener una primera referencia sobre el rendimiento de cada operación aplicada. Como desde la propia aplicación se puede modificar el dispositivo de procesamiento (CPU o GPU), pueden hacerse pequeñas comparativas de rendimiento de forma sencilla e interactiva, lo que puede ser de gran utilidad. En la Figura 38 se muestra un ejemplo de esta funcionalidad, donde se observa la aplicación de ciertas operaciones en ambos dispositivos de ejecución y los tiempos correspondientes.

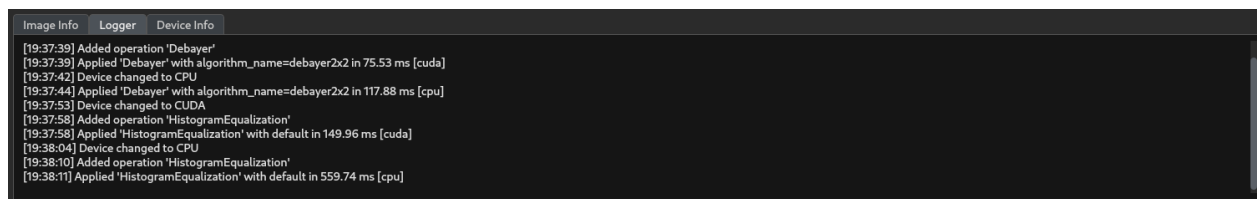


Figura 38: Cambio de dispositivo de procesamiento y referencia de tiempos de ejecución

Finalmente, la última de las pestañas (*Device Info*) muestra información que *PyTorch* proporciona sobre CUDA [37] y las GPUs disponibles en el sistema, como puede observarse en la Figura 39.

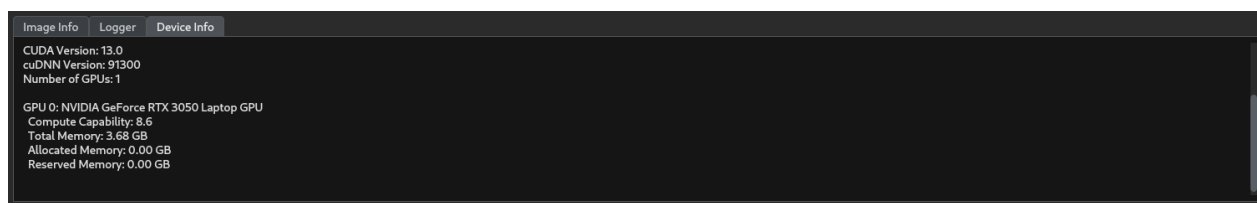


Figura 39: Pestaña de *Device Info* del visor central

8.3.5.3 Panel derecho (*RightPanel*)

El panel derecho se encarga de renderizar controles sobre las operaciones aplicadas desde el panel izquierdo para modificar y ajustar sus parámetros. Cada vez que se añade una nueva operación al *pipeline* de procesamiento, se añade un control específico para dicha operación en el panel derecho. En la Figura 40 se muestra su disposición respecto al resto de la interfaz, así como un ejemplo de los controles renderizados para un *pipeline* en específico.

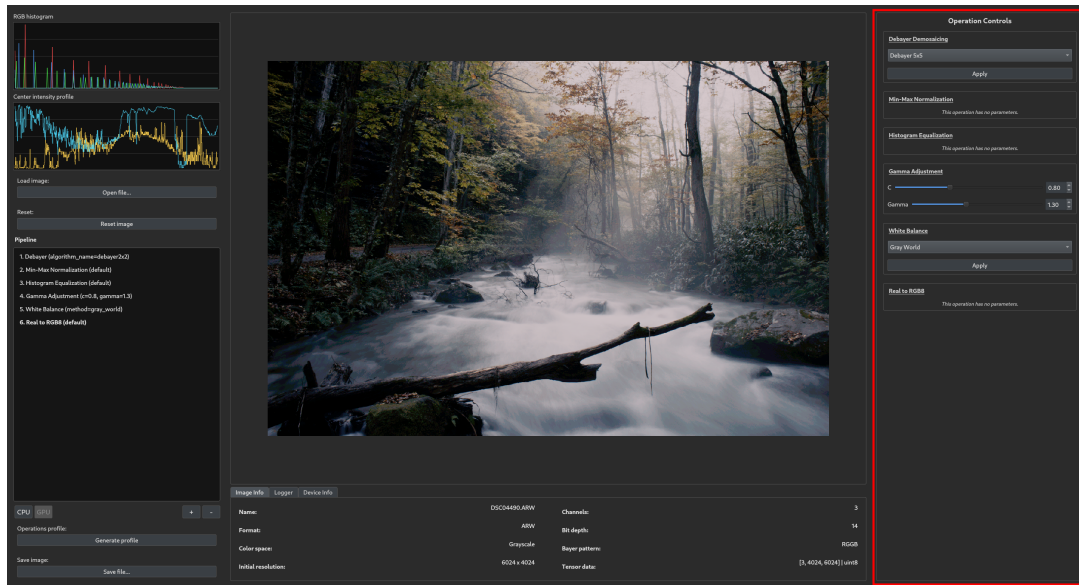
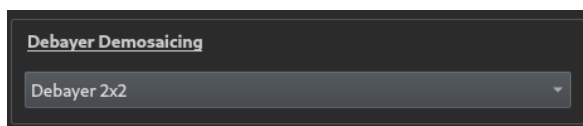


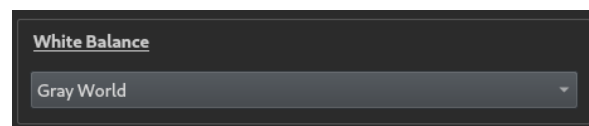
Figura 40: Panel derecho (*RightPanel*)

Desde estos controles, el usuario puede modificar los parámetros de cada operación aplicada, independientemente del orden en el que se encuentren en el *pipeline*. Como cada operación tiene sus propios parámetros, se han desarrollado diferentes tipos de controles para cada caso.

El primer tipo de control es el específico para operaciones que requieren la selección de tipo de algoritmo o método a aplicar, esto es, una selección entre varias opciones predefinidas. Este es el caso de la operación *Debayer*, que tiene diferentes módulos implementados, y de la operación de Balance de Blancos (*White Balance*), que permite la selección entre diferentes algoritmos. Dicho control, muestra un simple menú desplegable de tipo *dropdown* con las opciones disponibles para cada operación, como se muestra en la Figura 41.



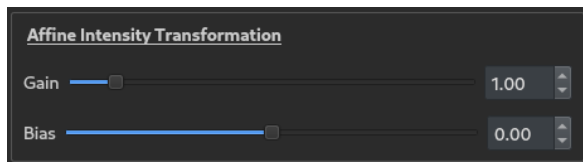
(a) Control para operación *Debayer*



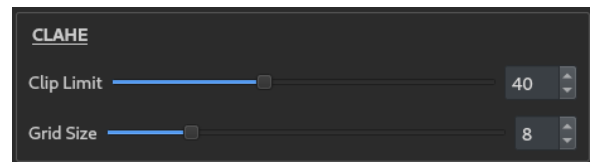
(b) Control para operación *White Balance*

Figura 41: Controles para operaciones con selección de método o algoritmo

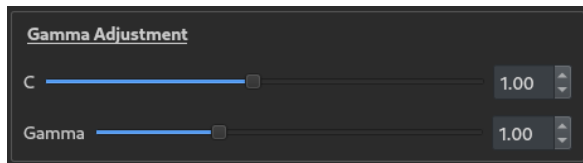
Otro tipo de control es el específico para operaciones que requieren la modificación de uno o varios parámetros numéricos. En este caso, se renderizan de forma dinámica elementos de tipo *slider* o *spinbox* para cada parámetro de la operación, permitiendo su ajuste de forma interactiva. Cada cambio en cualquiera de estos elementos implica la actualización inmediata de la imagen en el visor central, mostrando el resultado del ajuste. En la Figura 42 se muestran todas las operaciones que utilizan este tipo de control.



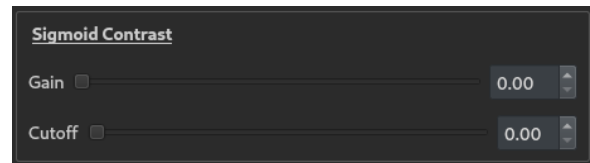
(a) Control para Ajuste de Intensidad Afín



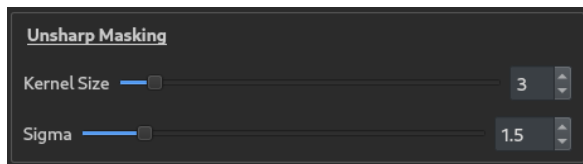
(b) Control para *CLAHE*



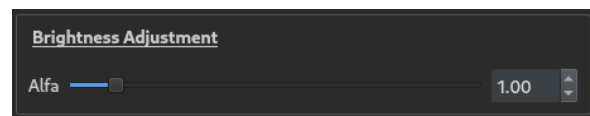
(c) Control para Ajuste *Gamma*



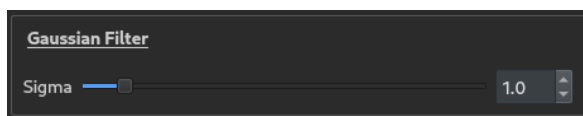
(d) Control para Ajuste de Contraste mediante Sigmoide



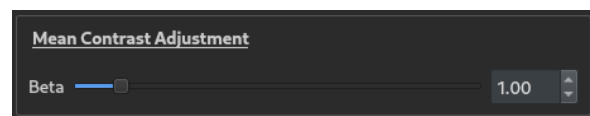
(e) Control para *Unsharp Masking*



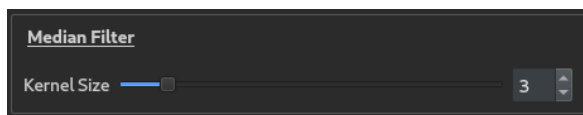
(f) Control para Ajuste de Brillo



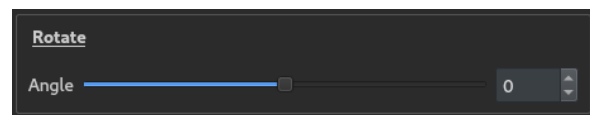
(g) Control para Filtro Gaussiano



(h) Control para Ajuste de Contraste



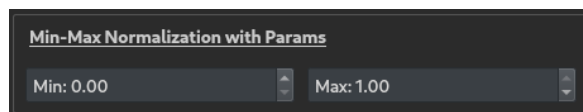
(i) Control para Filtro de Mediana



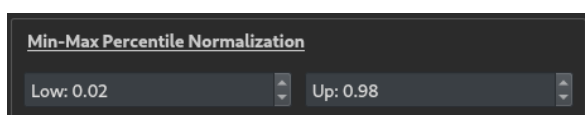
(j) Control para Rotación de imagen

Figura 42: Controles para operaciones con parámetros numéricos ajustables

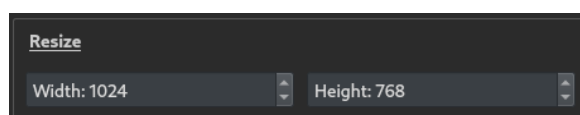
Similar al caso anterior, se implementa otro tipo de control para operaciones que requieren la modificación de dos parámetros numéricos para un rango de valores. Este es el caso de las operaciones mostradas en la Figura 43. Estos controles renderizan dos elementos de tipo *spinbox* para cada parámetro, permitiendo su ajuste de forma interactiva.



(a) Control para Normalización con parámetros



(b) Control para Normalización con percentiles



(c) Control para Redimensionamiento

Figura 43: Controles para operaciones con parámetros numéricos ajustables (continuación)

Otras operaciones requieren otros tipos de parámetros, por lo que se han desarrollado otros controles específicos. Este es el caso de la operación de volteo de imagen (*Flip*), que requiere el *toggle* entre dos opciones (volteo horizontal o vertical). Como puede observarse en la Figura 44, este control renderiza un *radio button* que permite seleccionar entre ambas opciones.

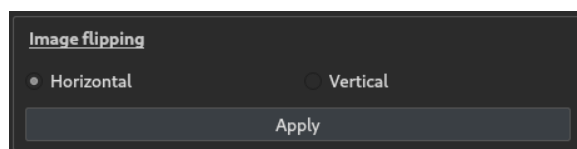


Figura 44: Control para Volteo de imagen

Otro caso es el de la operación de Compensación de Iluminación (*Light Compensation*), que requiere la carga de un fichero *numpy* para especificar la matriz de compensación y el ajuste de un parámetro numérico para la intensidad. Para manejar esta operación, se desarrolla el control mostrado en la Figura 45, que renderiza un botón para cargar el fichero de compensación y un *slider* para ajustar la intensidad de la compensación aplicada a la imagen.

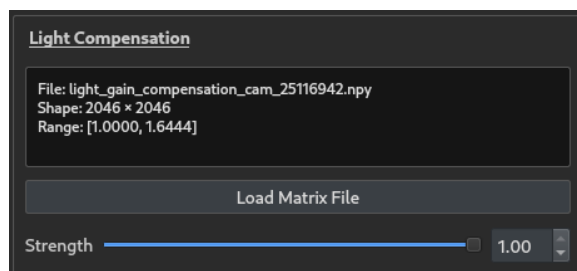
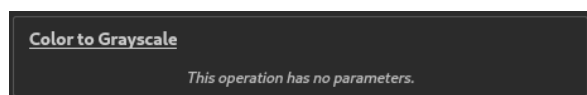
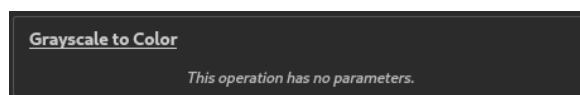


Figura 45: Control para Compensación de Iluminación

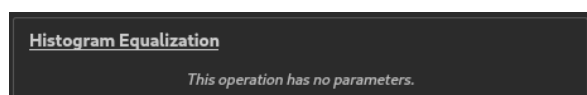
Finalmente, existen operaciones que no requieren ningún tipo de parámetro, simplemente su aplicación implica la ejecución de un proceso específico sobre la imagen, como puede ser el caso de una conversión de formato o espacio de color, etc. Para este tipo de operaciones, se renderiza un control muy simple que muestra el nombre de la operación aplicada y un texto informativo que indica que no requiere ningún parámetro, como se muestra en la Figura 46.



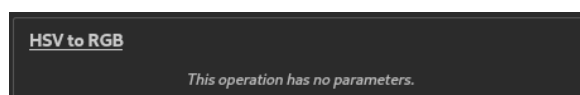
(a) Control para Conversión a Escala de Grises



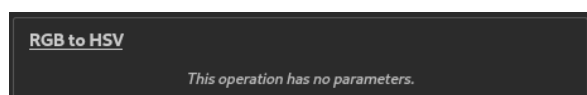
(b) Control para Conversión a Color



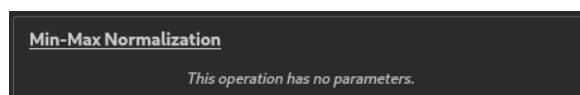
(c) Control para Ecualización de Histograma



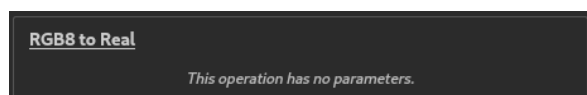
(d) Control para Conversión HSV a RGB



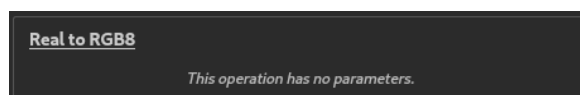
(e) Control para Conversión RGB a HSV



(f) Control para Normalización mín. máx.⁴



(g) Control para Conversión RGB8 a Real



(h) Control para Conversión Real a RGB8

Figura 46: Controles para operaciones sin parámetros

⁴La operación de Normalización mín. máx. (Figura 46f) sin parámetros se corresponde con la aplicación de esta técnica utilizando el valor mínimo y máximo de la imagen como parámetros, es decir, sin especificar manualmente ningún valor. Esta operación también puede aplicarse especificando manualmente los parámetros de mínimo y máximo, lo que se corresponde con la operación mostrada en la Figura 43a.

8.3.6 Benchmark

El componente *Benchmark* se encarga de medir y analizar el rendimiento de los *pipelines* de operaciones definidos por el usuario desde la *GUI*. Su implementación se basa en *pytest* [38] y en el *plugin* *pytest-benchmark* [39], lo que permite ejecutar medidas de rendimiento de forma repetitiva y con salida estructurada en formato *json*.

8.3.6.1 Arquitectura interna

El módulo se organiza alrededor de dos piezas principales: un archivo de configuración (*benchmark/-conftest.py*) y un archivo de ejecución de pruebas de rendimiento (*benchmark/run.py*). Esta organización es típica de *pytest* y permite separar la configuración general del *benchmark* de la lógica específica de ejecución. Las funcionalidades clave del módulo incluyen:

- **Configuración global del *benchmark*.** En *conftest.py* se definen parámetros comunes de ejecución (unidad temporal en milisegundos, *warmup* activo, criterio de ordenación y tiempo máximo por caso).
- **Perfil de ejecución externo.** El módulo añade el argumento de línea de comandos `--bench-profile`, que recibe la ruta de un fichero *json* con la definición del *pipeline* a evaluar.
- **Construcción dinámica del *pipeline*.** Mediante una *fixture* (*pipeline_factory*), cada entrada del perfil se transforma en una instancia real de operación utilizando el registro global de operaciones (*OPERATION_REGISTRY*, Párrafo 8.3.3.2).
- **Matriz experimental parametrizada.** En *run.py*, el *benchmark* se ejecuta para múltiples combinaciones de dispositivo de procesamiento, tamaño de imagen y tipo de dato, permitiendo comparar rendimiento y coste computacional en diferentes escenarios.

8.3.6.2 Flujo técnico de ejecución

Para cada caso de la matriz experimental definida, el flujo es el siguiente:

1. Carga y validación del perfil *json* recibido por `--bench-profile`, que representa el *pipeline* de operaciones a evaluar.
2. Resolución de operaciones por nombre a través de *OPERATION_REGISTRY*.
3. Instanciación de cada operación con sus parámetros.
4. Construcción de una función *pipeline* que aplica secuencialmente todas las operaciones recogidas en el mismo.
5. Generación de un tensor sintético de entrada con las dimensiones y precisión numérica configuradas.
6. Ejecución repetida del *pipeline* mediante *pytest-benchmark* para obtener métricas temporales.
7. Persistencia de resultados en el directorio `.benchmarks/` para su análisis posterior.

8.3.6.3 Comunicación con el resto de la aplicación

El componente *Benchmark* es un componente especial porque no forma parte del flujo de ejecución normal de la aplicación interactiva, sino que se ejecuta de forma autónoma. La integración de dicho componente con el sistema completo se realiza mediante dos mecanismos: un contrato de datos compartido y la reutilización del mismo núcleo *Core* de operaciones. Dicha integración se basa en los siguientes principios:

- **Interfaz de intercambio (GUI ↔ Benchmark).** Desde la interfaz gráfica, el usuario exporta el *pipeline* actual; el *Controller* lo serializa como una lista ordenada de operaciones y parámetros en formato *json*. Ese fichero se convierte en la entrada directa del módulo *Benchmark*.
- **Consistencia funcional (Core compartido).** El *Benchmark* no reimplementa operaciones: instancia exactamente las mismas clases del componente *Core* usando *OPERATION_REGISTRY*. Lo que se mide es exactamente el mismo comportamiento que se ejecuta en la aplicación interactiva.
- **Desacoplamiento arquitectónico.** No existe dependencia directa entre *GUI* y *Benchmark* en tiempo de ejecución; la comunicación se realiza a través del perfil exportado. Esto reduce acoplamiento y permite ejecutar *benchmarks* de forma autónoma, incluso por lotes mediante *scripts* de ejecución.

8.3.7 Tests

El componente *Tests* proporciona un marco de validación automatizada para todas las operaciones de procesamiento implementadas en el módulo *Core*. Su implementación se basa en *pytest* [38], siguiendo un enfoque de pruebas unitarias que verifica el comportamiento correcto de cada operación de forma independiente.

8.3.7.1 Organización y estructura

La *suite* de pruebas replica la estructura de categorías del componente *Core*, organizándose en módulos temáticos que agrupan tests relacionados:

- **tests/bayer/** → Validación de algoritmos de *demosaicing*.
- **tests/color/** → Conversiones entre espacios de color.
- **tests/filters/** → Operaciones de filtrado y mejora de contraste.
- **tests/format/** → Conversiones de formato de datos.
- **tests/geometry/** → Transformaciones geométricas.
- **tests/normalization/** → Técnicas de normalización.

8.3.7.2 Infraestructura de *fixtures*

El módulo `tests/conftest.py` define un conjunto de *fixtures* compartidas que proporcionan utilidades reutilizables a toda la *suite*:

- **base_tensor** → Genera tensores sintéticos de prueba con forma [C, H, W] configurable y tipo de dato especificado (float o int). Permite crear entradas de test reproducibles de forma programática.
- **assert_tensors** → Compara dos tensores verificando igualdad de forma, tipo de dato y valores dentro de una tolerancia configurable. Facilita la validación de resultados numéricos con margen de error controlado.
- **assert_channels** → Verifica que un tensor de imagen tenga el número esperado de canales en la dimensión correspondiente. Útil para validar transformaciones que alteran la dimensionalidad (como conversiones de color).

8.3.7.3 Tipología de pruebas

Cada operación se somete a múltiples categorías de validación que verifican distintos aspectos de su implementación:

1. **Validación de parámetros.** Comprueba que el constructor de la operación rechace valores inválidos (tipos incorrectos, rangos fuera de límites, configuraciones inconsistentes) mediante la verificación de excepciones esperadas.
2. **Corrección funcional.** Valida que la operación produzca el resultado matemático esperado para entradas conocidas, comparando la salida real con valores de referencia calculados independientemente.

En situaciones más complejas o inusuales, se incluyen pruebas adicionales que verifican casos límite, comportamiento con entradas atípicas o validación de propiedades específicas. También, en escenarios más concreto, se ejecutan pruebas paramétricas utilizando la anotación `@pytest.mark.parametrize` para evaluar múltiples combinaciones de parámetros de forma sistemática.

8.4 Diagrama de paquetes

9 Benchmarking

10 Resultados

11 Pruebas

12 Conclusiones

A Manual de usuario

B *Stack* tecnológico

C Formatos de imagen cruda

Referencias

- [1] “The 60-Year History of Digital Image Sensors as Told by Those Involved.” Disponible en: <https://petapixel.com>. Accedido: 12-12-2025.
- [2] “Procesamiento digital de imágenes y su importancia.” Disponible en: <https://www.massscience.com>. Accedido: 12-12-2025.
- [3] “Color Space.” Disponible en: <https://developer.mozilla.org>. Accedido: 11-01-2026.
- [4] “Digitalización: Espacios de color.” Disponible en: <https://www.atc.uniovi.es>. Accedido: 20-12-2025.
- [5] “HSV and RGB.” Disponible en: <https://developer.tuya.com>. Accedido: 11-01-2026.
- [6] “L*a*b* Color Space.” Disponible en: <https://www.sensorinstruments.de>. Accedido: 11-01-2026.
- [7] “YCbCr Color Space: Understanding Its Role in Digital Photography.” Disponible en: <https://proedu.com>. Accedido: 11-01-2026.
- [8] “Image Formats.” Disponible en: <https://www.geeksforgeeks.org>. Accedido: 15-12-2025.
- [9] “RAW File Format.” Disponible en: <https://www.cambridgeincolour.com>. Accedido: 15-12-2025.
- [10] “Introduction to Bayer Filters.” Disponible en: <https://www.arrow.com>. Accedido: 15-12-2025.
- [11] “Demosaicing.” Disponible en: <https://www.sciencedirect.com>. Accedido: 12-01-2026.
- [12] “Fundamentals of Digital Image Processing.” Disponible en: <https://www.academia.edu>. Accedido: 23-12-2025.
- [13] “Camera Histograms: tones and contrast.” Disponible en: <https://www.cambridgeincolour.com/>. Accedido: 23-12-2025.
- [14] “Imaging: Artifacts.” Disponible en: <https://www.imatest.com>. Accedido: 16-01-2026.
- [15] “Fundamentos de imagen digital aplicados a radiología.” Disponible en: <https://epos.myesr.org>. Accedido: 20-12-2025.
- [16] “Histogram Equalization.” Disponible en: <https://nikhilgandudi.medium.com>. Accedido: 16-01-2026.
- [17] “Adaptive Histogram Equalization.” Disponible en: <https://en.wikipedia.org>. Accedido: 16-01-2026.
- [18] “Histogram equalization CLAHE algorithm.” Disponible en: <https://medium.com>. Accedido: 16-01-2026.

- [19] “Normalization (image processing).” Disponible en: <https://en.wikipedia.org>. Accedido: 17-01-2026.
- [20] “Gamma Correction.” Disponible en: <https://en.wikipedia.org>. Accedido: 16-01-2026.
- [21] “Spatial filtering.” Disponible en: <https://fiveable.me>. Accedido: 17-01-2026.
- [22] “Gaussian Filter.” Disponible en: <https://docs.nvidia.com>. Accedido: 17-01-2026.
- [23] “Median Filter.” Disponible en: <https://docs.nvidia.com>. Accedido: 17-01-2026.
- [24] “Salt and Pepper Noise.” Disponible en: <https://en.wikipedia.org>. Accedido: 17-01-2026.
- [25] “Mean Filter.” Disponible en: <https://homepages.inf.ed.ac.uk>. Accedido: 17-01-2026.
- [26] “Sharpening.” Disponible en: <https://www.imatest.com>. Accedido: 17-01-2026.
- [27] “Converting RGB to HSV.” Disponible en: <https://mattlockyer.github.io>. Accedido: 17-01-2026.
- [28] “DSteps involved in an image processing pipeline.” Disponible en: <https://www.educative.io>. Accedido: 18-01-2026.
- [29] “PyTorch.” Disponible en: <https://pytorch.org/>. Accedido: 18-01-2026.
- [30] “torch.Tensor.” Disponible en: <https://pytorch.org>. Accedido: 18-01-2026.
- [31] “torch.dtype.” Disponible en: <https://pytorch.org>. Accedido: 18-01-2026.
- [32] N. Kehtarnavaz and M. Gamadia, *Real-Time Image and Video Processing*. Morgan and Claypool Publishers, 1ª ed., 2006.
- [33] “Model-View-Controller (MVC) Pattern.” Disponible en: <https://www.geeksforgeeks.org>. Accedido: 31-01-2026.
- [34] “dataclasses — Data Classes.” Disponible en: <https://docs.python.org>. Accedido: 01-02-2026.
- [35] “abc — Abstract Base Classes.” Disponible en: <https://docs.python.org>. Accedido: 01-02-2026.
- [36] “PyQt6 Documentation.” Disponible en: <https://pypi.org/project/PyQt6/>. Accedido: 03-02-2026.
- [37] “CUDA.” Disponible en: <https://developer.nvidia.com>. Accedido: 03-02-2026.
- [38] “pytest.” Disponible en: <https://docs.pytest.org>. Accedido: 01-03-2026.
- [39] “pytest-benchmark.” Disponible en: <https://pytest-benchmark.readthedocs.io>. Accedido: 01-03-2026.